
Unitary Convolutions for Message-passing and Positional Encodings on Directed Graphs

Lukas Fesser¹ Bobak Kiani² Melanie Weber¹

Abstract

In many real-world networks, relationships are inherently directional, yet most graph neural networks (GNNs) assume undirected edges, and naïve adaptations of undirected GNNs to directed graphs amplify oversmoothing and gradient pathologies that cap model depth. Unitary graph convolutions (UniConv) provably prevent representational collapse and oversmoothing, but do not incorporate edge directionality or edge features. In this paper, we introduce a **directed unitary** GNN with **edge** features (**Dune**), which retains these guarantees while overcoming UniConv’s limitations by incorporating edge directionality and edge features. Dune keeps gradient norms bounded at any number of layers, allowing it to benefit from neural network depth, unlike existing directed GNNs. The same unitary operator can be embedded in hybrid architectures with graph transformers, where its wave-like propagation supplies positional information and reduces the importance of random-walk or Laplacian-based encodings. We prove that Dune avoids exponential oversmoothing that plagues existing directed GNNs, and empirically show that it achieves state-of-the-art performance on 12 directed-graph benchmarks while remaining trainable beyond 100 layers, improving performance by up to 18 percentage points over baselines. Our results establish unitary convolutions as a scalable, geometry-aware foundation for deep learning on directed graphs. Code is available [here](#).

1. Introduction

Machine learning methods for graph-structured data have achieved remarkable success across various domains, yet

¹Harvard University ²Bowdoin College. Correspondence to: Melanie Weber <mweber@seas.harvard.edu>.

most existing graph neural networks (GNNs) assume undirected edges and symmetric information propagation. In numerous real-world applications—including communication networks, transportation systems, citation graphs, and electrical circuits—relationships between entities are inherently directional (Easley & Kleinberg, 2010; Newman, 2010). Ignoring such directionality discards essential information about signal, influence, or resource flows within networks, and naïve adaptations of undirected architectures typically exacerbate known issues rather than resolve them (Rossi et al., 2024).

Extending GNNs to directed graphs remains challenging. Common strategies include treating each directed edge as two opposing undirected edges, but this significantly increases computational costs without effectively differentiating source from target nodes during message passing (Tong et al., 2020). Other methods incorporate minor modifications to convolutional or attention mechanisms, often resulting in ad hoc solutions that neither ensure stable training nor resolve fundamental architectural issues (Jiang et al., 2025). More recently, transformer-based approaches have emerged, explicitly designed to capture directional information through positional encodings. These positional encodings are usually precomputed quantities such as random walk transition probabilities or Laplacian eigenvectors that are meant to capture the geometry of a directed graph (Geisler et al., 2023; Huang et al., 2024b).

However, despite the promise of transformer-based architectures with advanced positional encodings, existing approaches still face significant hurdles. Common positional encodings require careful hyperparameter tuning and - as in the undirected case - scale poorly with the size of the graph (Dwivedi et al.; Kreuzer et al., 2021). The flexibility and scalability of these models is therefore limited. Most existing message-passing based GNNs (MPNNs) for directed graphs similarly suffer from architectural shortcomings, such as gradient instabilities or oversmoothing, where node representations become indistinguishable after several layers (Li et al., 2018; Alon & Yahav, 2021). Consequently, even state-of-the-art directed MPNNs often plateau in performance after relatively few layers, and cannot benefit from modeling long-range dependencies (Cai & Wang, 2020).

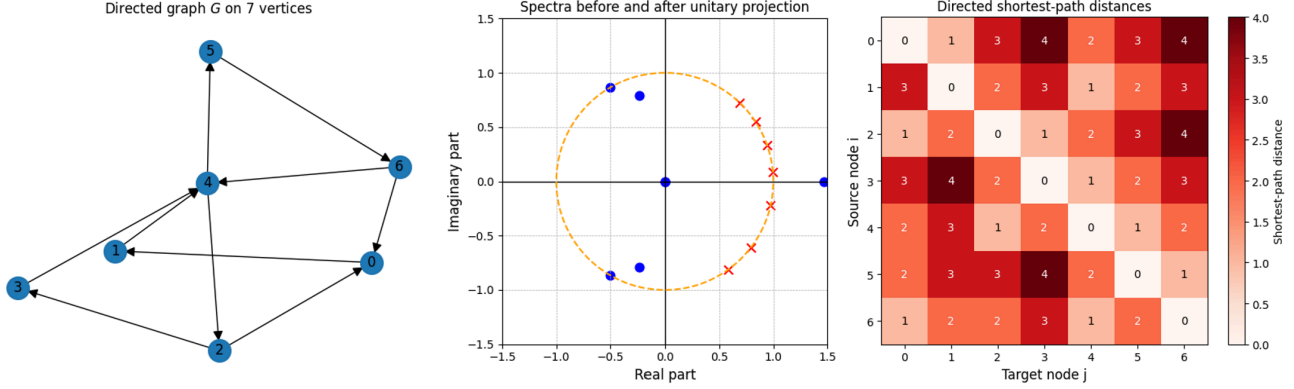


Figure 1. An example directed graph G (left), the spectrum of its adjacency matrix before (blue) and after the mapping into the unitary group (red) that Dune employs (center), and the directed shortest distances between the nodes in G (right). Dune excels at capturing these topological features.

In this paper, we introduce a directed unitary GNN with edge features (Dune), which builds on the recently proposed Unitary Graph Convolutional Neural Network (UniGCN) (Kiani et al., 2024). Dune extends UniGCN by incorporating edge directionality and rich edge features. By leveraging unitary transformations, Dune provably avoids oversmoothing and ensures gradient norms remain bounded regardless of network depth. Dune can also be shown to capture rich geometric information about an input graph, which can reduce the importance of positional encodings when using transformers on directed graphs. Empirically, these properties allow Dune to set a new state of the art for tasks on directed graphs.

Contributions. Our key contributions in this paper are:

1. We propose Dune, a powerful unitary graph neural network that supports directed graphs and graphs with edge features.
2. We provide a theoretical analysis of oversmoothing on directed graphs and show that Dune provably avoids this. Vanishing and exploding gradients can similarly be ruled out with Dune.
3. We introduce the use of unitary convolutions in hybrid models, i.e. with graph transformers. We empirically and theoretically show that unitary convolutions encode rich geometric information that can reduce the importance of positional encodings.
4. Finally, we present experimental results on 12 directed graph datasets that show that Dune significantly outperforms the state of the art when used for message-passing or with graph transformers. Ablation studies show that unlike existing approaches, our model is still trainable with more than 100 layers.

2. Preliminaries

We use c , w , and M to denote scalars, vectors, and matrices, respectively. For a matrix M , its transpose and conjugate transpose are written $M^\top$ and $M^\dagger$. For matrices A and B , their tensor product is $A \otimes B$. For a vector w , the Euclidean norm is $\|w\|$, while for a matrix M , we use $\|M\|$ for the operator norm and $\|M\|_F$ for the Frobenius norm.

Graph Neural Networks We write graphs as $\mathcal{G} = (V, E)$ with V and E denoting the sets of nodes and edges. For a graph on n nodes, unless stated otherwise, we set $V = \{1, \dots, n\}$ and denote its adjacency matrix by $A \in \mathbb{R}^{n \times n}$ and node features by $X \in \mathbb{R}^{n \times d}$. The diagonal (undirected) degree matrix $D \in \mathbb{R}^{n \times n}$ has entries D_{ii} equal to the degree of node i , and the normalized adjacency is $\tilde{A} = D^{-1/2} A D^{-1/2}$. On directed graphs, we denote the in-degree and out-degree matrices by D_{in} and D_{out} , respectively. Given node features $X \in \mathbb{R}^{n \times d_{\text{in}}}$, where row i stores the d_{in} -dimensional vector of node i , graph convolutions on undirected graphs take the general form (Kipf & Welling, 2017; Gilmer et al., 2017)

$$f_{\text{conv}}(X; A) = XW_0 + AXW_1 + \dots + A^k XW_k, \quad (1)$$

with trainable parameters $W_0, \dots, W_k \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. For simplicity, we often omit A as an explicit argument. In practice, one usually includes a single “message passing” step, yielding $f_{\text{conv}}(X) = AXW$. Graph convolutions are permutation equivariant, since for any permutation matrix $P_\pi \in \mathbb{R}^{n \times n}$,

$$f_{\text{conv}}(P_\pi X; P_\pi A P_\pi^{-1}) = P_\pi f_{\text{conv}}(X; A). \quad (2)$$

Group Theory Basics Symmetries (“invariances”) are transformations that leave data properties unchanged and thus capture the geometric structure of the domain. Algebraically, they are modeled by groups. A group is a matrix

Model	Directionality	Edge Features	Expressivity	No Oversmoothing	PE Connection
Di-GCN	✓	✗	✗	✗	✗
MagNet	✓	✗	✗	✗	✗
Dir-GNN	✓	✓	✓	✗	✗
NDDGNN	✓	✗	✗	✗	✗
UniGCN	✗	✗	✗	✓	✗
Dune (Ours)	✓	✓	✓	✓	✓

Table 1. Comparison of (directed) GNN models

Lie group if it is a closed subgroup of the set $GL(n, \mathbb{R})$ of invertible $n \times n$ matrices (Stillwell, 2008). Each Lie group has an associated Lie algebra, the tangent space at the identity. For an introduction, see (Hall, 2015). In this work we encounter the orthogonal $O(n)$ and unitary $U(n)$ groups,

$$O(n) = \{U \in \mathbb{R}^{n \times n} : UU^\top = I\},$$

$$U(n) = \{U \in \mathbb{C}^{n \times n} : UU^\dagger = I\}.$$

Their Lie algebras consist of skew-symmetric and skew-Hermitian matrices,

$$\mathfrak{o}(n) = \{M \in \mathbb{R}^{n \times n} : M + M^\top = 0\},$$

$$\mathfrak{u}(n) = \{M \in \mathbb{C}^{n \times n} : M + M^\dagger = 0\}.$$

For $M \in \mathfrak{o}(n)$ (or $\mathfrak{u}(n)$), the matrix exponential maps it into the group, i.e., $\exp(M) \in O(n)$ (or $U(n)$). (Kiani et al., 2024) use the exponential map to define a graph convolutional operator on undirected graphs as in equation 1 that is also unitary:

Definition 1. (Separable unitary graph convolution (UniConv)). Given an undirected graph G over n nodes with adjacency matrix $A \in \mathbb{R}^{n \times n}$, separable unitary graph convolution (UniConv) $f_{\text{Uconv}} : \mathbb{C}^{n \times d} \rightarrow \mathbb{C}^{n \times d}$ takes the form

$$f_{\text{Uconv}}(\mathbf{X}) = \exp(iAt)\mathbf{X}U, \quad UU^\dagger = I, \quad (3)$$

where $U \in U(d)$ is a unitary operator and $t \in \mathbb{R}$ controls the magnitude of the convolution.

Note that since A is a symmetric matrix, $\exp(iAt)$ is unitary for all values of $t \in \mathbb{R}$ and corresponds to vanilla message passing up to first order: $\exp(iAt) \approx I + iAt + O(t^2)$.

3. Related Work

GNNs for directed graphs. Graph Neural Networks for directed graphs have evolved rapidly to capture edge asymmetry beyond simply treating digraphs as undirected. One major line of work extends message-passing neural networks (MPNNs) by separately aggregating incoming and outgoing messages: Dir-GNN performs two distinct neighborhood aggregations, provably matches the expressivity of the Directed Weisfeiler–Lehman test, and yields large gains

on heterophilic benchmarks without harming homophilic performance (Rossi et al., 2024). Other models such as DGCN and Di-GCN define directed Laplacians or PPR-based convolutions to incorporate edge direction (Tong et al., 2020; Zheng et al., 2021). Finally, complex-valued approaches such as MagNet employ a Hermitian adjacency (the Magnetic Laplacian) to encode directionality in the phases of complex embeddings (Zhang et al., 2021).

Positional encodings, graph transformers, and hybrid models. By default, transformers treat their inputs as sets, which in the case of graphs means that they can use the node features, but not the topology to make predictions (Kreuzer et al., 2021; Dwivedi et al.). To remedy this, graph transformers are commonly equipped with positional encodings (PEs), which provide the node features with information about the graph topology. While there is an abundance of positional encodings for undirected graphs (Fesser & Weber, 2023; Rampásek et al., 2022; Dwivedi et al.; Kreuzer et al., 2021), relatively few encodings have been developed for the directed case. Most prominent among those are PEs based on the transition probabilities of random walks (Geisler et al., 2023) or based on the spectral decomposition of the Magnetic-Laplacian (Geisler et al., 2023; Huang et al., 2024b). PEs and transformers specifically for directed acyclic graphs (DAGs) have also been developed (Luo et al., 2023). Hybrid models are graph transformers that use message-passing GNNs to capture geometric information to complement or even replace positional encodings. Examples of these in the undirected setting include GraphGPS (Dwivedi et al.), Exphormer (Shirzad et al., 2023), and GRIT (Ma et al., 2023). Notably, these models use PEs and MP-GNNs to capture geometric information. Recent work has investigated to what extent MP-GNNs can be used to learn the same geometric information as existing PEs, although these insights are limited to the undirected case (Kanatoulis et al., 2025).

Unitary neural networks. Unitary neural network architectures trace their roots to early efforts at stabilizing deep and recurrent models by enforcing norm-preserving transformations. Initial works applied unitary or orthogonal constraints to RNN weight matrices—using techniques like parameterizations in the Lie algebra or structured matrix

factorizations—to mitigate vanishing and exploding gradients and thus capture long-range dependencies in sequence data (Arjovsky et al., 2016; Wisdom et al., 2016; Henaff et al., 2016). Extensions of these ideas to convolutional networks introduced orthogonal and unitary convolutions for image data, often via exponential-map parameterizations of circulant or Lie-group operators, yielding improved training stability in deep CNNs (Sedghi et al., 2018; Li et al., 2019; Trockman & Kolter, 2021; Singla & Feizi, 2021). More recently, graph neural network research has adopted unitary message-passing schemes to prevent representational collapse (oversmoothing) and maintain gradient norms without auxiliary interventions like skip connections or batch normalization (Guo et al., 2022; Qiu et al., 2024; Kiani et al., 2024).

4. Dune

The key challenge in adapting unitary convolutions to directed graphs is that their adjacency matrix is in general not symmetric. Consequently, $i\mathbf{A}t$ is no longer skew Hermitian for general $t \in \mathbb{R}$, so $\exp(i\mathbf{A}t)$ may not be unitary. Our formulation of Dune works around this while also incorporating edge features into the convolution operation. We draw inspiration from (Lezcano-Casado & Martinez-Rubio, 2019) who first proposed a similar Hermitian projection to the one in equation 4, although not in the context of graph machine learning.

4.1. Model Definition

Definition 2. (Directed unitary GNN with edge features (Dune)). For an arbitrary (not necessarily symmetric) adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, we consider Hermitian projections of the form

$$\Pi(\mathbf{A}) = i\frac{\alpha}{2}(\mathbf{A} + \mathbf{A}^\dagger) + \frac{\beta}{2}(\mathbf{A} - \mathbf{A}^\dagger), \quad (4)$$

which is in the Lie algebra of the unitary group for any $\alpha, \beta \in \mathbb{R}$, as a simple calculation shows. In the following we set $\alpha = \beta = \frac{1}{2}$ and perform the unitary convolution as

$$f_{\text{conv}}(\mathbf{X}; \mathbf{A}) = \exp(\Pi(\mathbf{A})t)\mathbf{X}\mathbf{W}, \quad (5)$$

given a scaling factor $t > 0$ and weights $\mathbf{W} \in \mathbb{C}^{d_{\text{in}} \times d_{\text{out}}}$. $\exp$ here denotes the matrix exponential. For real-valued prediction tasks, we project the complex-valued representation back to real-valued features; we use the real component in our experiments and discuss this choice and its relation to expressivity in Appendix G.4. Now, note that a Taylor expansion around $t = 0$ gives

$$\begin{aligned} \exp(\Pi(\mathbf{A})t) &= \mathbf{I} + \Pi(\mathbf{A})t + \mathcal{O}(t^2) \\ &= \mathbf{I} + \frac{i}{4}(\mathbf{A} + \mathbf{A}^\dagger)t + \frac{1}{4}(\mathbf{A} - \mathbf{A}^\dagger)t + \mathcal{O}(t^2). \end{aligned} \quad (6)$$

so a Dune layer sends messages in both directions of a directed edge while still retaining directional information. Assume now that the edges E of the input graph $G = (V, E)$ have edge features $E_{ij} \in \mathbb{R}^{d_E}$. To incorporate these into the convolution operation, we construct an edge-weighted adjacency matrix. First, we choose a potentially trainable map $f_E : \mathbb{R}^{d_E} \rightarrow \mathbb{C}$ and then construct the matrix

$$[A_E]_{ij} = \begin{cases} f_E(E_{ij}) & \text{if } (i, j) \text{ is an edge,} \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

Then, we perform unitary convolution with the map $\Pi(A_E)$. Unless explicitly stated otherwise, we use a simple linear layer to parametrize f_E .

This design keeps the unitary propagator lightweight, but it also compresses each edge feature vector into a single complex scalar, i.e., two real degrees of freedom per edge. Thus, for tasks where edges carry rich semantic, relational, or geometric embeddings, this edge-to-scalar map may become a bottleneck. Higher-capacity variants could instead use multi-channel edge generators $f_E : \mathbb{R}^{d_E} \rightarrow \mathbb{C}^r$, relation-specific propagators, or tensorized edge-to-generator maps, at an r -dependent computational cost. We leave these extensions to future work.

We implement the exponential map in equation 5 using a Taylor approximation. Empirically, more than ~ 10 terms are never necessary since the approximation error decreases exponentially (Kiani et al., 2024). On many datasets, the order of the Taylor expansion truncation can be reduced while maintaining stability; see Appendix D).

4.2. Dune as an MPNN

Dune as defined above comes with several theoretical guarantees that make it attractive as a stand-alone message-passing model. We present these results here and defer all proofs to Appendix B. To begin with, a model using Dune layers can be made as expressive as an extension of the 1-WL test to directed graphs (D -WL) (Grohe et al., 2021).

Proposition 3 (Dune Expressivity (informal)). *Consider a directed GNN constructed using the Dune updates in equation 5, where the weight matrices $\mathbf{W}^{(k)}$ are injective for all layers k and where any non-linearities σ used after equation 5 and the final readout are all injective. Then the model is as expressive as D -WL.*

The proof of this follows from the Taylor expansion in equation 6 and an argument similar to the one presented in (Rossi et al., 2024) for Dir-GNN. From the same paper, it follows that Dune is more powerful than message-passing neural networks that operate on either an undirected version of the graph or that propagate information along only one direction of a directed edge. Unlike Dir-GNN, Dune also

comes with guarantees against oversmoothing. Intuitively, oversmoothing describes the phenomenon that the features of neighboring nodes become more similar as the number of network layers increases. This is usually measured via the Dirichlet Energy (Maskey et al., 2023), which can be defined on directed (and strongly-connected) graphs as follows.

Definition 4 (Dirichlet energy). Let G be a directed graph with adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. Set diagonal degree matrices $\mathbf{D}_{\text{in}} = \text{diag}(d_i^{\text{in}})$ and $\mathbf{D}_{\text{out}} = \text{diag}(d_i^{\text{out}})$ (assume $d_i^{\text{in}}, d_i^{\text{out}} > 0$). Define $\mathbf{S} := \frac{1}{2} (\mathbf{D}_{\text{in}}^{-1/2} \mathbf{A} \mathbf{D}_{\text{out}}^{-1/2} + \mathbf{D}_{\text{out}}^{-1/2} \mathbf{A}^\top \mathbf{D}_{\text{in}}^{-1/2})$ and

$$\mathbf{H}_{\text{dir}} := \frac{1}{4} (\mathbf{D}_{\text{out}} \mathbf{D}_{\text{in}}^{-1} + \mathbf{D}_{\text{in}} \mathbf{D}_{\text{out}}^{-1}) - \frac{1}{2} \mathbf{S}. \quad (8)$$

Then for any feature matrix $\mathbf{X} \in \mathbb{C}^{n \times d}$, the Dirichlet energy is

$$\mathcal{E}(\mathbf{X}) := \frac{1}{4} \sum_{i,j} \mathbf{A}_{ij} \left\| \frac{\mathbf{x}_i}{\sqrt{d_i^{\text{in}}}} - \frac{\mathbf{x}_j}{\sqrt{d_j^{\text{out}}}} \right\|_2^2 = \text{Tr}(\mathbf{X}^\dagger \mathbf{H}_{\text{dir}} \mathbf{X}). \quad (9)$$

The normalized Dirichlet energy then takes the form

$$\bar{\mathcal{E}}(\mathbf{X}) := \mathcal{E} \left(\frac{\mathbf{X}}{\|\mathbf{X}\|_F} \right) = \frac{\text{Tr}(\mathbf{X}^\dagger \mathbf{H}_{\text{dir}} \mathbf{X})}{\|\mathbf{X}\|_F^2}. \quad (10)$$

Definition 5 (Oversmoothing). Let $\{\mathbf{X}^{(k)}\}_{k \geq 0}$ be the layer by layer features of a GNN on a directed graph, and set $\bar{E}_k := \bar{\mathcal{E}}(\mathbf{X}^{(k)})$ to be the normalized Dirichlet energy at layer k as in Definition 4. The model oversmooths on the given features if there exist constant $c > 0$ and integer $k_0 > 0$ such that for all layers $k \geq k_0$,

$$\bar{E}_k - \lambda_{\min}(\mathbf{H}_{\text{dir}}) \leq e^{-ck} (\bar{E}_0 - \lambda_{\min}(\mathbf{H}_{\text{dir}})). \quad (11)$$

Like its UniGCN cousin on undirected graphs, Dune can be shown to not suffer from oversmoothing, which opens the way for much deeper models in applications.

Proposition 6 (Dune never oversmooths). Consider a depth K directed GNN constructed using the Dune updates in equation 5 without nonlinearities, where the weight matrices $\mathbf{W}^{(k)}$ are unitary for all layers k . Then for any initial node features $\mathbf{X}^{(0)}$ that do not already minimize the Rayleigh quotient at initialization, the K -layer Dune model with the update rule in equation 5 does **not** oversmooth.

Dune is therefore an exception among directed graph neural networks. Many other architectures display oversmoothing in experiments, and for some this undesirable behavior can even be proven (DeZoort & Hanin, 2023; Chen et al., 2025). The following proposition illustrates this with the example of DiGCN (Tong et al., 2020). Figure 2 compares this and Dir-GCN (Rossi et al., 2024) experimentally with Dune.

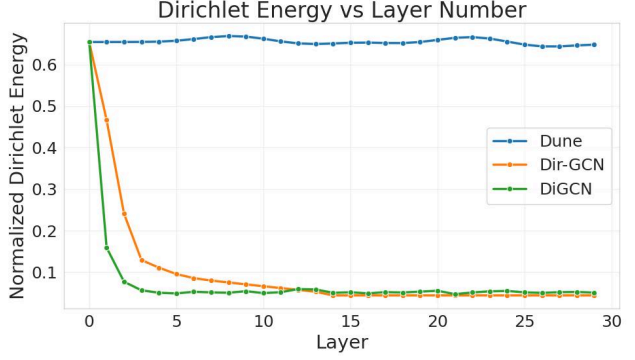


Figure 2. Dirichlet energy comparison on a synthetic Erdos-Renyi graph, generated as in (Geisler et al., 2023).

Proposition 7 (DiGCN oversmooths). Let $\alpha \in (0, 1)$, let $\mathbf{P} = (1 - \alpha) \mathbf{P}_{\text{rw}} + \frac{\alpha}{n} \mathbf{1} \mathbf{1}^\top$ be the PageRank transition matrix with stationary distribution π , and let $\Pi = \text{Diag}(\pi)$, where $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector. Here $\mathbf{P}_{\text{rw}} = \mathbf{D}^{-1} \mathbf{A}$ is the row-stochastic random-walk matrix built from the adjacency matrix. Define the DiGCN update rule as

$$\begin{aligned} \mathbf{S} &= \frac{1}{2} (\Pi^{1/2} \mathbf{P} \Pi^{-1/2} + \Pi^{-1/2} \mathbf{P}^\top \Pi^{1/2}), \\ \mathbf{X}^{(k+1)} &= \mathbf{S} \mathbf{X}^{(k)} \Theta^{(k)}. \end{aligned} \quad (12)$$

Assume that $\inf_{k \geq 0} \|\Pi_{\min} \mathbf{X}^{(0)} \mathbf{B}_k\|_F > 0$, where $\mathbf{B}_k := \Theta^{(0)} \Theta^{(1)} \dots \Theta^{(k-1)}$ and $\Pi_{\min}$ is the orthogonal projector onto the eigenspace corresponding to $\lambda_{\min}(\mathbf{H}_{\text{dir}})$. Then the resulting K -layer DiGCN oversmooths on all Eulerian digraphs (strongly-connected digraphs where all nodes have the same in- and out-degree).

Finally, and in contrast to conventional GNNs, where the product of contractive Jacobians leads to vanishing gradients (and, less commonly, to exploding ones when spectral norms exceed unity) (Álvaro Arroyo et al., 2025), the Dune update rule 5 guarantees norm preservation at every layer. During backpropagation, the gradient with respect to parameters at layer i involves the product Jacobian

$$\mathbf{J} = \prod_{k=i+1}^K \frac{\partial \mathbf{X}^{(k)}}{\partial \mathbf{X}^{(k-1)}}, \quad (13)$$

whose spectral norm determines whether gradients vanish, explode, or remain stable. Since both the propagator $\exp(t\Pi(\mathbf{A}))$ and the weight matrices $\mathbf{W}^{(k)}$ are unitary, their operator norms equal one, and thus the layer-wise Jacobians $\mathbf{J}_k$ satisfy $\|\mathbf{J}_k\|_2 = 1$. Consequently, the gradient norm across K layers is exactly preserved, $\|\mathbf{J}\|_2 = 1$, ruling out the exponential decay ($\lambda < 1$) or blow-up ($\lambda > 1$) familiar from standard message-passing networks (at least in the absence of nonlinearities (Arjovsky et al., 2016)). This property shows that vanishing and exploding gradients, typically

caused by spectral contraction or expansion in the adjacency and weight matrices, are provably absent in Dune.

4.3. Dune in a Hybrid Architecture

We now leave the pure message-passing setting and focus on using Dune in a hybrid model with a graph transformer. As mentioned in section 3, hybrid architectures use MP-GNNs that can update node features with (local) geometric information using one or more message-passing layers. A global attention layer then uses these updated node features as input to model global dependencies (Shirzad et al., 2023; Rampášek et al., 2022) (see Appendix A.3 for more details). In this subsection, we therefore aim to explain which geometric information Dune can capture particularly well for a given input graph. We begin with the following generalization of a directed walk between a pair of nodes.

Definition 8 (Bidirectional Walk). Let $G = (V, E)$ be a directed graph. A *bidirectional walk* $w = (v_0, v_1, v_2, \dots)$ is a sequence of nodes where every consecutive pair of nodes forms either a forward edge $(v_i, v_{i+1}) \in E$ or a backward edge $(v_{i+1}, v_i) \in E$. The *length* of a bidirectional walk is the total number of forward and backward edges it contains.

Definition 9 (Walk Profile). Let G be a directed graph with adjacency matrix A . Given two nodes $u, v \in V$, the *walk profile* $\Phi_{u,v}(\ell, k)$ is the number of length- ℓ bidirectional walks from u to v that contain exactly k forward edges and $\ell - k$ backward edges.

Using the Magnetic Laplacian, Huang et al. (2024b) introduce a theoretically-grounded positional encoding for directed graphs that provably captures the bidirectional walk profile. To permit the exact recovery of all bidirectional walk counts up to length ℓ , this Multi- q Magnetic-Laplacian spectral encoding requires $Q \geq \lceil \ell/2 \rceil + 1$ eigendecompositions, which results in a complexity of $O(Q|V|^3)$ (see appendix A.4 for details). We now state informally that Dune captures information about the walk profile in sub-cubic time. Sections B and C in the appendix elaborate further on this.

Proposition 10 (Dune aggregates information about walk profiles (informal)). *A depth- K Dune model whose exponential map is implemented via an order T Taylor approximation aggregates weighted sums of bidirectional walk counts up to length $\leq KT$. On directed acyclic graphs, this recovers the bidirectional walk profiles up to a scalar multiple.*

5. Experiments

In this section, we experimentally demonstrate the effectiveness of unitary convolutions on directed graphs on a variety of tasks, including node classification and graph regression. In particular, we aim to answer the following questions:

1. Does Dune improve message-passing on directed graphs? And if so, is this mainly due to using unitary convolutions or due to incorporating edge directionality?
2. How does Dune perform as part of a hybrid architecture with graph transformers? Can the information captured by Dune reduce the importance of existing positional encodings, as our theoretical analysis suggests?

In addition to our main experiments, we provide an extensive set of ablations in both the main text and the appendix, which we discuss more at the end of the section.

5.1. Experimental design

To address question 1, we compare Dune against standard MP-GNNs on undirected graphs, as well as state-of-the-art MP-GNNs on directed ones. We conduct an extensive hyperparameter search (see Appendix G for details) and report the numbers from the original papers whenever they are available and better than the best performance we obtain. For undirected convolutions such as GCN, we convert the input graph into an undirected graph before training. For all models, we record the test set accuracy of the settings with the highest validation accuracy. We accumulate experimental results across 10 random trials and report the mean test accuracy, along with the 95% confidence interval.

For question 2, we follow (Geisler et al., 2023; Huang et al., 2024b) and equip the Structure-Aware Transformer (SAT) (Chen et al., 2022) with different MP-GNN backbones and either no positional encoding ('None') or one of Lap, Maglap computed with only one q value, or the Maglap-Multi- q encoding mentioned in the last section. We report numbers from (Huang et al., 2024b) whenever possible and strictly follow their experimental evaluation protocol and hyperparameter tuning when not. We report the average root mean squared error (RMSE) across 10 random trials along with the 95% confidence interval.

5.2. Datasets

The node-classification experiments draw on seven directed benchmarks, spanning homophilic citation networks (Cora, Citeseer (Bojchevski & Günnemann, 2017), OGBN-Arxiv (Hu et al., 2020)), citation network datasets (Arxiv-Year, Snap-Patents (Lim et al., 2021)), and strongly heterophilic graphs (Roman-Empire, Questions (Platonov et al., 2023)). For the Questions dataset we report ROC-AUC, for all other datasets we report accuracy. We evaluate each with its original node features and published train/validation/test splits. Further details can be found in Appendix G.3.

For the hybrid transformer-GNN studies, we use the Open Circuit Benchmark datasets (Dong et al., 2023). These

Model	Cora	Citeseer	OGBN-Arxiv	Arxiv-Year	SNAP-P.	Roman E.	Questions
GCN	84.37 $\pm$ 1.52	93.37 $\pm$ 0.22	68.39 $\pm$ 0.11	46.28 $\pm$ 0.39	51.02 $\pm$ 0.07	56.23 $\pm$ 0.37	76.09 $\pm$ 1.27
SAGE	86.01 $\pm$ 1.56	94.15 $\pm$ 0.61	67.78 $\pm$ 0.17	44.05 $\pm$ 0.22	52.55 $\pm$ 0.10	72.05 $\pm$ 0.41	76.44 $\pm$ 0.62
GAT	86.44 $\pm$ 1.45	94.53 $\pm$ 0.48	69.60 $\pm$ 0.26	45.30 $\pm$ 0.23	54.28 $\pm$ 0.14	49.18 $\pm$ 1.35	77.43 $\pm$ 1.20
UniGCN	86.49 $\pm$ 1.56	71.13 $\pm$ 0.68	68.85 $\pm$ 0.21	55.32 $\pm$ 0.22	64.67 $\pm$ 0.12	87.21 $\pm$ 0.54	79.21 $\pm$ 0.81
Di-GCN	87.35 $\pm$ 1.06	92.75 $\pm$ 0.75	65.41 $\pm$ 0.12	54.13 $\pm$ 0.18	59.86 $\pm$ 0.14	54.71 $\pm$ 0.32	75.82 $\pm$ 0.73
MagNet	85.26 $\pm$ 1.05	93.38 $\pm$ 0.89	67.22 $\pm$ 0.14	60.29 $\pm$ 0.27	71.33 $\pm$ 0.15	88.07 $\pm$ 0.27	77.67 $\pm$ 0.78
Dir-GCN	84.45 $\pm$ 1.69	93.44 $\pm$ 0.59	66.66 $\pm$ 0.12	64.08 $\pm$ 0.26	73.95 $\pm$ 0.06	84.54 $\pm$ 0.71	75.94 $\pm$ 1.36
Dir-SAGE	85.84 $\pm$ 2.09	94.14 $\pm$ 0.65	65.14 $\pm$ 0.13	61.76 $\pm$ 0.10	70.26 $\pm$ 0.14	91.23 $\pm$ 0.32	77.43 $\pm$ 0.85
Dir-GAT	86.21 $\pm$ 1.40	94.48 $\pm$ 0.52	66.50 $\pm$ 0.16	62.47 $\pm$ 0.14	73.82 $\pm$ 0.08	82.25 $\pm$ 0.14	77.19 $\pm$ 1.12
NDDGNN	88.14 $\pm$ 1.28	94.17 $\pm$ 0.58	68.76 $\pm$ 0.32	65.02 $\pm$ 0.32	74.15 $\pm$ 0.04	91.76 $\pm$ 0.27	77.58 $\pm$ 1.24
Dune (Ours)	87.82 $\pm$ 1.14	94.39 $\pm$ 0.83	72.16 $\pm$ 0.28	65.86 $\pm$ 0.10	75.12 $\pm$ 0.14	92.58 $\pm$ 0.22	80.45 $\pm$ 0.88

Table 2. Undirected (top rows) and directed (bottom rows) message-passing GNNs’ performance on directed node classification datasets. Details on baselines can be found in Appendix A.

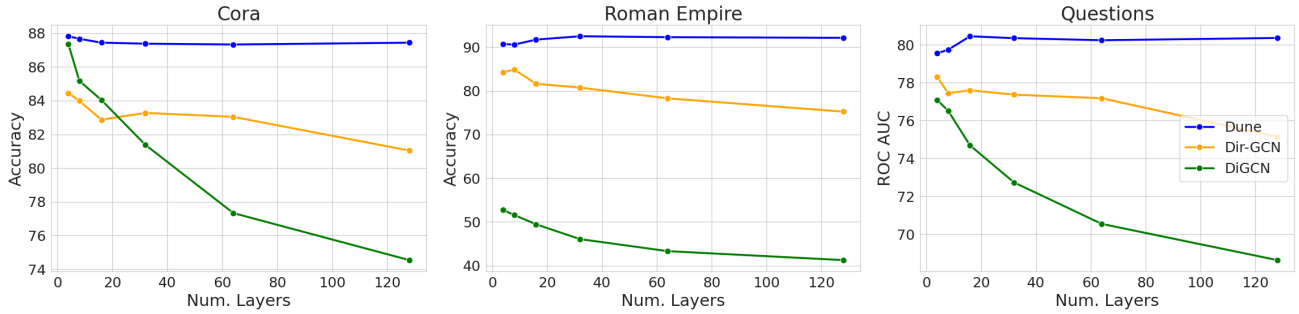


Figure 3. Node classification performance at deeper layers. Higher is better.

contain 10,000 operational amplifier circuits as directed graphs. The task is to predict the DC gain (Gain), band width (BW) and phase margin (PM) of each circuit. We expect directionality to be useful here because these targets reflect the property of current flows from input nodes to output nodes. The dataset consists of 2-stage amplifiers and 3-stage amplifiers. We follow (Huang et al., 2024b) and use 2-stage amplifiers, which we randomly split 0.9/0.05/0.05 into train/ val/ test. Also for the hybrid models, we use the HLS dataset (Wu et al., 2022) which contains 18,750 intermediate representation (IR) graphs of C/C++ code after front-end compilation (Aho et al., 2006). The dataset provides post-implementation performance metrics on FPGA devices as labels. The task is to predict resource usage: look-up table (LUT) and digital signal processor (DSP) usage. We again follow (Huang et al., 2024b) and randomly select 16570 for training, and 1000 each for validation and testing.

5.3. Results

We present our experimental results in Tables 2 and 3. Using these, we can answer our original two questions as follows:

1. Dune’s superior performance over existing directed GNNs directly reflects the value added by unitary convolutions on top of edge directionality. Across all seven benchmarks, replacing standard message-passing with

unitary operations yields consistent gains, with the largest benefits on OGBN-Arxiv (from 68.76 % to 72.16 %) and on Questions (77.67 to 80.45). These improvements show that unitary convolutions consistently enhance feature propagation once edge directions are known. Conversely, comparing Dune to UniGCN isolates the benefit of incorporating directionality: performance here improves significantly, i.e. from 64.67 to 75.12 on Snap-Patents, and from 87.21 to 92.58 on Roman-Empire. We also note that the gains from using Dune are more significant on large datasets, such as Snap or OGBN, while on small datasets such as Cora and Citeseer, many models are within reach of each other. On these two homophilous datasets, a certain amount of oversmoothing might even be beneficial.

2. When combined with the SAT transformer architecture (Chen et al., 2022), Dune turns out to be a much better message-passing backbone than GIN - both directed and undirected - and also outperforms the undirected UniGCN on all five datasets. We also note that with a Dune backbone, the benefits derived from adding positional encodings are much smaller than with other backbones and often become statistically insignificant. This indicates that Dune already captures relevant geometric information, as our theoretical results suggest.

Unitary Convolutions for Message-passing and Positional Encodings on Directed Graphs

Model	PE method	Gain	BW	PM	DSP	LUT
SAT (undirected-GIN)	None	0.431 ± 0.023	4.362 ± 0.134	1.068 ± 0.036	3.242 ± 0.205	2.492 ± 0.172
	Lap	0.375 ± 0.016	4.180 ± 0.093	1.065 ± 0.034	3.167 ± 0.193	2.425 ± 0.168
	Maglap-1q (best q)	0.361 ± 0.016	4.014 ± 0.068	1.057 ± 0.036	3.101 ± 0.176	2.362 ± 0.154
	Maglap-Multi-q	0.350 ± 0.004	4.044 ± 0.153	1.035 ± 0.025	3.076 ± 0.240	2.333 ± 0.147
SAT (bidirected-GIN)	None	0.423 ± 0.018	4.278 ± 0.102	1.162 ± 0.038	2.789 ± 0.142	2.231 ± 0.117
	Lap	0.368 ± 0.022	4.024 ± 0.106	1.046 ± 0.021	2.713 ± 0.135	2.173 ± 0.107
	Maglap-1q (best q)	0.360 ± 0.009	3.960 ± 0.060	1.062 ± 0.024	2.657 ± 0.128	2.107 ± 0.135
	Maglap-Multi-q	0.359 ± 0.008	3.930 ± 0.069	1.045 ± 0.012	2.616 ± 0.151	2.082 ± 0.099
SAT (UniGCN)	None	0.388 ± 0.019	4.235 ± 0.098	1.112 ± 0.030	2.891 ± 0.149	2.315 ± 0.126
	Lap	0.369 ± 0.014	4.095 ± 0.088	1.068 ± 0.023	2.835 ± 0.145	2.271 ± 0.124
	Maglap-1q (best q)	0.361 ± 0.012	4.041 ± 0.074	1.063 ± 0.021	2.785 ± 0.140	2.228 ± 0.121
	Maglap-Multi-q	0.349 ± 0.011	4.020 ± 0.072	1.057 ± 0.019	2.754 ± 0.132	2.205 ± 0.114
SAT (Dune)	None	0.355 ± 0.010	3.970 ± 0.064	1.028 ± 0.018	2.612 ± 0.111	2.021 ± 0.093
	Lap	0.352 ± 0.009	3.945 ± 0.060	1.025 ± 0.016	2.570 ± 0.108	2.015 ± 0.092
	Maglap-1q (best q)	0.350 ± 0.009	3.932 ± 0.059	1.023 ± 0.015	2.558 ± 0.106	2.007 ± 0.091
	Maglap-Multi-q	0.345 ± 0.008	3.925 ± 0.058	1.012 ± 0.015	2.552 ± 0.105	1.951 ± 0.090

Table 3. Graph regression results with hybrid models (SAT + message-passing backbone) and positional encodings. We report mean RMSE (lower is better) and standard deviations over 10 independent runs.

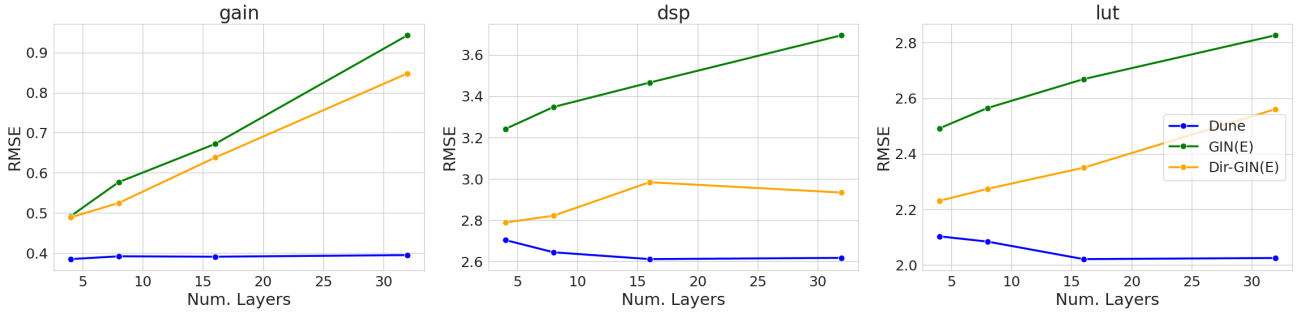


Figure 4. Graph regression performance at deeper layers. Lower is better.

Benefits from depth are especially pronounced on the DSP and LUT datasets: Dune performance improves while non-unitary models’ performance drops (Figure 4). We observe similar trends on the node classification in Figure 3, where we see that Dune performance on heterophilous datasets (Roman Empire, Questions) increases with depth.

Additional results using more hybrid architectures, other node classification baselines including graph transformers, and ablations on the importance of skip connections and normalization techniques when using unitary layers can be found in Appendix F. While our focus in the main text has been on Dune’s performance on directed graphs, we also provide ablations on (undirected) graphs with edge features in Appendix F.5. Here too, architectures with unitary message-passing layers significantly outperform commonly-used non-unitary layers, such as GINe or GatedGCN. Further results on synthetic datasets that investigate to what extent Dune captures geometric information can be found in Appendix F.4. Finally, we investigate DUNE’s runtime complexity and memory consumption in Appendix C to

show that DUNE is comparable to well-established models such as Dir-GAT.

6. Discussion

In this paper, we introduced Dune, a unitary graph convolutional network specifically designed for directed graphs and graphs with edge features, capable of addressing critical problems such as oversmoothing and gradient instabilities that have limited the depth and expressivity of previous directed GNN architectures. Dune leverages norm-preserving, unitary transformations to propagate messages in a manner that prevents representational collapse, ensuring stable training even at considerable depths. Our empirical evaluation demonstrates that Dune consistently outperforms or equals existing models on 12 benchmark datasets, highlighting its effectiveness not only as stand-alone message-passing GNN but also in hybrid architectures involving graph transformers. Importantly, we showed that Dune implicitly encodes geometric information that typically requires PEs.

Limitations and future work. Dune’s main limitation is its scalability to extremely large graphs on the order of SNAP or larger. It shares this limitation with existing models, such as DiGCN, MagNet, and Dir-GAT. We therefore believe that GNNs that work well on directed graphs of this extreme size should be investigated further. We are also convinced that connecting unitary convolutions and positional encodings is a promising area for future work. In particular, one might want to use a unitary GNN to explicitly learn PEs in the style of the recently introduced PEARL framework (Kanatsoulis et al., 2025), instead of relying on the inductive biases that come with a unitary GNN in a hybrid architecture. Finally, we believe that directed link prediction, being a significantly more challenging task than node classification or graph regression (He et al., 2025) requires a separate investigation.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

LF was supported by a Kempner Institute Graduate Fellowship. BTK and MW were supported by NSF award CBET-2112085. MW further acknowledges support from an Alfred P. Sloan Fellowship in Mathematics and DARPA under agreement no. HR0011-25-3-0205.

References

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, August 2006. ISBN 0321486811. URL <http://www.amazon.ca/exec/obidos/redirect?tag=citeulike09-20&path=ASIN/0321486811>.
- Alon, U. and Yahav, E. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021.
- Arjovsky, M., Shah, A., and Bengio, Y. Unitary evolution recurrent neural networks. In *ICML*, 2016.
- Bojchevski, A. and Günnemann, S. Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. *arXiv preprint arXiv:1707.03815*, 2017.
- Cai, C. and Wang, Y. A note on over-smoothing for graph neural networks. *arXiv preprint arXiv:2006.13318*, 2020.
- Chen, D., O’Bray, L., and Borgwardt, K. Structure-aware transformer for graph representation learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, Proceedings of Machine Learning Research, 2022.
- Chen, Z., Lin, Z., Chen, S., Polyanskiy, Y., and Rigollet, P. Residual connections provably mitigate oversmoothing in graph neural networks, 2025. URL <https://arxiv.org/abs/2501.00762>.
- DeZoort, G. and Hanin, B. Principles for initialization and architecture selection in graph neural networks with relu activations, 2023. URL <https://arxiv.org/abs/2306.11668>.
- Dong, Z., Cao, W., Zhang, M., Tao, D., Chen, Y., and Zhang, X. Cktgcn: Circuit graph neural network for electronic design automation. *arXiv preprint arXiv:2308.16406*, 2023.
- Dwivedi, V. P., Luu, A. T., Laurent, T., Bengio, Y., and Bresson, X. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations*.
- Dwivedi, V. P., Rampásek, L., Galkin, M., Parviz, A., Wolf, G., Luu, A. T., and Beaini, D. Long range graph benchmark. *Advances in Neural Information Processing Systems*, 35:22326–22340, 2022.
- Easley, D. and Kleinberg, J. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.
- Fesser, L. and Weber, M. Effective structural encodings via local curvature profiles. *arXiv preprint arXiv:2311.14864*, 2023.
- Geisler, S., Li, Y., Mankowitz, D. J., Cemgil, A. T., Günnemann, S., and Paduraru, C. Transformers meet directed graphs. In *International conference on machine learning*, pp. 11144–11172. PMLR, 2023.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *ICML*, 2017.
- Gómez-Bombarelli, R., Wei, J. N., Duvenaud, D., Hernández-Lobato, J. M., Sánchez-Lengeling, B., Sheberla, D., Aguilera-Iparraguirre, J., Hirzel, T. D., Adams, R. P., and Aspuru-Guzik, A. Automatic chemical design using a data-driven continuous representation of molecules. *ACS central science*, 4(2):268–276, 2018.
- Grohe, M., Kersting, K., Mladenov, M., and Schweitzer, P. Color refinement and its applications. 2021. URL <https://api.semanticscholar.org/CorpusID:59069015>.

- Guo, K., Zhou, K., Hu, X., Li, Y., Chang, Y., and Wang, X. Orthogonal graph neural networks. In *AAAI*, 2022.
- Hall, B. C. *Lie Groups, Lie Algebras, and Representations: An Elementary Introduction*. Springer, 2015.
- Hamilton, W., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- He, M., Guo, Y., Zheng, Y., Wei, Z., Günnemann, S., and Xiao, X. Rethinking link prediction for directed graphs. *arXiv preprint arXiv:2502.05724*, 2025.
- Henaff, M., Szlam, A., and LeCun, Y. Recurrent orthogonal networks and long-memory tasks. In *ICML*, 2016.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.
- Huang, J., Mo, Y., Hu, P., Shi, X., Yuan, S., Zhang, Z., and Zhu, X. Exploring the role of node diversity in directed graph representation learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pp. 2072–2080, 2024a.
- Huang, Y., Wang, H., and Li, P. What are good positional encodings for directed graphs? *arXiv preprint arXiv:2407.20912*, 2024b.
- Jiang, Q., Wang, C., Lones, M., and Pang, W. Demystifying mpnns: Message passing as merely efficient matrix multiplication. *arXiv preprint arXiv:2502.00140*, 2025.
- Kanatsoulis, C. I., Choi, E., Jegelka, S., Leskovec, J., and Ribeiro, A. Learning efficient positional encodings with graph neural networks. *arXiv preprint arXiv:2502.01122*, 2025.
- Kiani, B., Fesser, L., and Weber, M. Unitary convolutions for learning on graphs and groups. *Advances in Neural Information Processing Systems*, 37:136922–136961, 2024.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- Kreuzer, D. et al. Rethinking graph transformers with spectral attention. In *NeurIPS*, 2021.
- Lezcano-Casado, M. and Martínez-Rubio, D. Cheap orthogonal constraints in neural networks: A simple parametrization of the orthogonal and unitary group. In *International Conference on Machine Learning*, pp. 3794–3803. PMLR, 2019.
- Li, Q., Han, Z., and Wu, X.-M. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Li, Q., Haque, S., Anil, C., Lucas, J., Grosse, R. B., and Jacobsen, J.-H. Preventing gradient attenuation in lipschitz constrained convolutional networks. In *NeurIPS*, 2019.
- Lim, D., Hohne, F., Li, X., Huang, S. L., Gupta, V., Bhalerao, O., and Lim, S. N. Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods. *Advances in neural information processing systems*, 34:20887–20902, 2021.
- Luo, Y., Thost, V., and Shi, L. Transformers over directed acyclic graphs, 2023. URL <https://arxiv.org/abs/2210.13148>.
- Ma, L., Lin, C., Lim, D., Romero-Soriano, A., Dokania, P. K., Coates, M., Torr, P., and Lim, S.-N. Graph inductive biases in transformers without message passing. In *International Conference on Machine Learning*, pp. 23321–23337. PMLR, 2023.
- Maskey, S., Paolino, R., Bacho, A., and Kutyniok, G. A fractional graph laplacian approach to oversmoothing. *arXiv preprint arXiv:2305.13084*, 2023.
- Newman, M. E. J. *Networks: An Introduction*. Oxford University Press, 2010.
- Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., and Prokhorenkova, L. A critical look at the evaluation of gnns under heterophily: Are we really making progress? *arXiv preprint arXiv:2302.11640*, 2023.
- Qiu, H., Bian, Y., and Yao, Q. Graph unitary message passing. *arXiv preprint arXiv:2403.11199*, 2024.
- Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems*, 35:14501–14515, 2022.
- Rossi, E., Charpentier, B., Di Giovanni, F., Frasca, F., Günnemann, S., and Bronstein, M. M. Edge directionality improves learning on heterophilic graphs. In *Learning on graphs conference*, pp. 25–1. PMLR, 2024.
- Sedghi, H., Gupta, V., and Long, P. M. The singular values of convolutional layers. In *ICLR Workshop*, 2018.
- Shirzad, H., Velingker, A., Venkatachalam, B., Sutherland, D. J., and Sinop, A. K. Expformer: Sparse transformers for graphs. In *International Conference on Machine Learning*, pp. 31613–31632. PMLR, 2023.

- Singla, S. and Feizi, S. Skew orthogonal convolutions. In *NeurIPS*, 2021.
- Stillwell, J. *Naive Lie Theory*. Springer, 2008.
- Tong, Z., Liang, Y., Sun, C., Li, X., Rosenblum, D., and Lim, A. Digraph inception convolutional networks. *Advances in Neural Information Processing Systems*, 33, 2020.
- Trockman, A. and Kolter, J. Z. Orthogonalizing convolutional layers with the cayley transform. In *ICML*, 2021.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., and Bengio, Y. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- Wisdom, S., Powers, T., Hershey, J. R., Le Roux, J., and Atlas, L. Full-capacity unitary recurrent neural networks. In *NeurIPS*, 2016.
- Wu, N., Yang, H., Xie, Y., Li, P., and Hao, C. High-level synthesis performance prediction using gnns: Benchmarking, modeling, and advancing. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pp. 49–54, 2022.
- Zhang, X., He, Y., Brugnone, N., Perlmutter, M., and Hirn, M. Magnet: A neural network for directed graphs. *Advances in neural information processing systems*, 34: 27003–27015, 2021.
- Zheng, Y., Gao, C., Chen, L., Jin, D., and Li, Y. Dgcnn: Diversified recommendation with graph convolutional networks. In *Proceedings of the Web Conference 2021*, pp. 401–412, 2021.
- Álvaro Arroyo, Gravina, A., Gutteridge, B., Barbero, F., Gallicchio, C., Dong, X., Bronstein, M., and Vandergheynst, P. On vanishing gradients, over-smoothing, and over-squashing in gnns: Bridging recurrent and graph learning, 2025. URL <https://arxiv.org/abs/2502.10818>.

A. Further Related Literature

A.1. Graph Neural Networks on Undirected Graphs

Graph Convolutional Network (GCN) (Kipf & Welling, 2017). Let $G = (V, E)$ be an undirected graph with adjacency matrix A and degree matrix $D = \text{diag}(A\mathbf{1})$; its combinatorial Laplacian is $L = D - A$. A GCN stack contains K layers acting on a node-feature matrix $X^{(0)} \in \mathbb{R}^{|V| \times d_0}$. For layer $k \in \{0, \dots, K-1\}$,

$$X^{(k+1)} = \sigma(\hat{A} X^{(k)} W^{(k)}), \quad \hat{A} = (D + I)^{-\frac{1}{2}}(A + I)(D + I)^{-\frac{1}{2}}, \quad (14)$$

where $W^{(k)} \in \mathbb{R}^{d_k \times d_{k+1}}$ is trainable and σ is a point-wise nonlinearity (e.g., ReLU). The composite operator $(\sigma \circ \hat{A})^K$ aggregates information over progressively larger neighborhoods while remaining permutation-equivariant.

Graph Attention Network (GAT) (Veličković et al., 2017). Each GAT layer replaces the fixed propagation matrix $\hat{A}$ with content-dependent attention coefficients. Given node states $x_u^{(k)}$, first compute $z_u = W^{(k)} x_u^{(k)}$. Shared attention parameters $a \in \mathbb{R}^{2d_{k+1}}$ yield unnormalized logits $e_{uv} = \text{LeakyReLU}(a^\top [z_u \| z_v])$ for every edge $(u, v) \in E$ and the self-loop (v, v) . Neighborhood-wise soft-max produces $\alpha_{uv} = \exp(e_{uv}) / \sum_{w \in \mathcal{N}(v)} \exp(e_{vw})$. The k -th layer update is then

$$x_v^{(k+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} z_u\right), \quad (15)$$

optionally averaged across multiple attention heads, allowing the network to focus dynamically on the most relevant neighbors.

GraphSAGE (Hamilton et al., 2017). GraphSAGE equips each node with an inductive neighborhood aggregator. For layer k , sample (or take) the neighbor set $\mathcal{N}(v)$ and compute $\text{AGG}^{(k)}(\mathcal{N}(v))$ using a chosen scheme (mean, max-pool, LSTM, etc.) over $\{x_u^{(k)} : u \in \mathcal{N}(v)\}$. The update rule is

$$x_v^{(k+1)} = \sigma\left(W^{(k)} [x_v^{(k)} \| \text{AGG}^{(k)}(\mathcal{N}(v))]\right), \quad (16)$$

followed (optionally) by ℓ_2 normalization $x_v^{(k+1)} \leftarrow x_v^{(k+1)} / \|x_v^{(k+1)}\|$. After K layers, the model yields scalable, permutation-equivariant representations that capture K -hop neighborhood structure without explicitly computing the Laplacian L .

A.2. Graph Neural Networks on Directed Graphs

DiGCN (Digraph Inception Convolutional Network) (Tong et al., 2020) For a directed graph $G = (V, E)$ with adjacency A , set $P = D^{-1}(A + I)$ (row-stochastic with self-loops) and let $\pi^\top$ be its stationary distribution with teleport probability α , so that $\Pi = \text{diag}(\pi)$. DiGCN builds an approximate personalized-PageRank magnetic Laplacian

$$L_{\text{appr}} = I - \frac{1}{2}(\Pi^{1/2} P \Pi^{-1/2} + \Pi^{-1/2} P^\top \Pi^{1/2}). \quad (17)$$

At layer k ($0 \leq k < K$) it forms multi-scale proximity matrices $P^{(0)} = I$, $P^{(1)} = P$, $P^{(r)}$ ($r \geq 2$) by intersecting forward and reverse r -hop walks, then computes

$$Z^{(r)} = \begin{cases} X^{(k)} \Theta_0^{(k)}, & r = 0, \\ \frac{1}{2}(\Pi^{1/2} P \Pi^{-1/2} + \Pi^{-1/2} P^\top \Pi^{1/2}) X^{(k)} \Theta_1^{(k)}, & r = 1, \\ W_r^{-1/2} P^{(r)} W_r^{-1/2} X^{(k)} \Theta_r^{(k)}, & r \geq 2, \end{cases} \quad (18)$$

with $W_r = \text{diag}(P^{(r)} \mathbf{1})$. An *Inception* fusion operator Γ (e.g., sum or concat-linear) combines $\{Z^{(r)}\}_{r=0}^R$, after which $X^{(k+1)} = \sigma(\Gamma(Z^{(0)}, \dots, Z^{(R)}))$. Stacking K layers yields variable-receptive-field, direction-aware convolutions while reserving the symbol L for other Laplacians.

MagNet (Zhang et al., 2021) Edge orientation is injected through the *magnetic Laplacian*

$$\mathbf{L}_N^{(q)} = \mathbf{I} - (\mathbf{D}_s^{-1/2} \mathbf{A}_s \mathbf{D}_s^{-1/2}) \odot \exp(i\Theta^{(q)}), \quad \Theta_{uv}^{(q)} = 2\pi q (\mathbf{A}_{uv} - \mathbf{A}_{vu}), \quad (19)$$

where $\mathbf{A}_s = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top)$ and $\mathbf{D}_s$ its degree; $q \in [0, 1/4]$ is learnable. Scaling to $\tilde{\mathbf{L}}^{(q)} \in [-1, 1]$, the k -th layer applies a Chebyshev filter of order R :

$$\mathbf{X}^{(k+1)} = \sigma\left(\sum_{r=0}^R T_r(\tilde{\mathbf{L}}^{(q)}) \mathbf{X}^{(k)} \Theta_r^{(k)}\right). \quad (20)$$

Complex ReLU activations maintain phase information, and the real/imaginary parts are concatenated after the final (K -th) layer.

Dir-GCN (Rossi et al., 2024). Dir-GCN augments message passing with separate inward and outward channels. Define normalized direction matrices $\mathbf{S}_{\rightarrow} = \mathbf{D}_{\text{out}}^{-1/2} \mathbf{A} \mathbf{D}_{\text{in}}^{-1/2}$ and $\mathbf{S}_{\leftarrow} = \mathbf{S}_{\rightarrow}^\top$. At layer k ,

$$\begin{aligned} \mathbf{M}_{\rightarrow}^{(k)} &= \mathbf{S}_{\rightarrow} \mathbf{X}^{(k)} \mathbf{W}_{\rightarrow}^{(k)}, \\ \mathbf{M}_{\leftarrow}^{(k)} &= \mathbf{S}_{\leftarrow} \mathbf{X}^{(k)} \mathbf{W}_{\leftarrow}^{(k)}, \end{aligned} \quad (21)$$

and the update is

$$\mathbf{X}^{(k+1)} = \sigma(\alpha \mathbf{M}_{\rightarrow}^{(k)} + (1 - \alpha) \mathbf{M}_{\leftarrow}^{(k)}), \quad \alpha \in [0, 1] \text{ (learnable or fixed)}. \quad (22)$$

Stacking K layers implicitly mixes higher-order operators such as $\mathbf{A}\mathbf{A}^\top$, enhancing robustness on heterophilic digraphs.

NDDGNN (Node-Diversity DGNN) (Huang et al., 2024a). NDDGNN refines Dir-GNN with node-specific gates that consider *neighbor* and *degree* diversity. For each node v at layer k compute Dirichlet energies $e_{\rightarrow}^{(k)}(v)$ and $e_{\leftarrow}^{(k)}(v)$, add learned degree embeddings, and apply a softmax to obtain diagonal gates $\Gamma_{\rightarrow}^{(k)}$ and $\Gamma_{\leftarrow}^{(k)} = \mathbf{I} - \Gamma_{\rightarrow}^{(k)}$. The update rule is

$$\mathbf{X}^{(k+1)} = \sigma\left(\alpha \mathbf{X}^{(k)} + \Gamma_{\rightarrow}^{(k)} \mathbf{S}_{\rightarrow} \mathbf{X}^{(k)} \mathbf{W}_{\rightarrow}^{(k)} + \Gamma_{\leftarrow}^{(k)} \mathbf{S}_{\leftarrow} \mathbf{X}^{(k)} \mathbf{W}_{\leftarrow}^{(k)}\right), \quad (23)$$

with a consistency regularizer keeping the gates well-behaved. After K layers the model yields direction-adaptive representations without overloading the Laplacian symbol $\mathbf{L}$.

A.3. Graph Transformers

Structure-Aware Transformer (SAT++) (Chen et al., 2022). SAT++ augments the vanilla multi-head Transformer encoder with graph-aware inductive biases tailored to code abstract-syntax trees (ASTs). A model comprises K identical layers. At layer k ($1 \leq k \leq K$) the current node embeddings $\mathbf{H}^{(k-1)} \in \mathbb{R}^{n \times d}$ are first processed by a lightweight, three-step, direction-aware GNN that mixes incoming and outgoing messages, yielding query and key bases $\tilde{\mathbf{Q}}^{(k)}, \tilde{\mathbf{K}}^{(k)}$, while the values remain $\mathbf{V}^{(k)} = \mathbf{H}^{(k-1)}$. For each attention head h linear projections give $\mathbf{Q}_h^{(k)} = \tilde{\mathbf{Q}}^{(k)} \mathbf{W}_{Q,h}$, $\mathbf{K}_h^{(k)} = \tilde{\mathbf{K}}^{(k)} \mathbf{W}_{K,h}$, and $\mathbf{V}_h^{(k)} = \mathbf{V}^{(k)} \mathbf{W}_{V,h}$. Attention weights are

$$\alpha_h^{(k)} = \text{softmax}\left(\frac{\mathbf{Q}_h^{(k)} \mathbf{K}_h^{(k)\top}}{\sqrt{d_h}} + \mathbf{M}\right), \quad (24)$$

where the additive mask $\mathbf{M}$ injects edge features and directional positional encodings derived from magnetic-Laplacian and random-walk spectra. Head outputs $\mathbf{O}_h^{(k)} = \alpha_h^{(k)} \mathbf{V}_h^{(k)}$ are concatenated, projected, and combined with a degree-scaled residual connection followed by LayerNorm to produce $\hat{\mathbf{H}}^{(k)}$. A position-wise two-layer feed-forward network with GeLU activation, sparse token dropout, and a second residual-norm pair yields the layer output $\mathbf{H}^{(k)}$. After K layers the embedding of the designated *function* node is fed to a linear classifier.

A.4. Positional Encodings on Directed Graphs

Magnetic-Laplacian spectral encodings (Geisler et al., 2023). Let $G = (V, E)$ be a directed graph with adjacency matrix $\mathbf{A} \in \{0, 1\}^{|V| \times |V|}$. Define the symmetrized adjacency $\mathbf{A}_s = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top)$ and its degree matrix $\mathbf{D}_s = \text{diag}(\mathbf{A}_s \mathbf{1})$. For a parameter $q \in [0, 1]$, the *Magnetic Laplacian* is

$$\mathbf{L}_U^{(q)} = \mathbf{D}_s - \mathbf{A}_s \odot \exp(i\Theta^{(q)}), \quad \Theta_{uv}^{(q)} = 2\pi q (\mathbf{A}_{uv} - \mathbf{A}_{vu}). \quad (25)$$

Because $L_U^{(q)}$ is Hermitian, its eigenvectors form an orthonormal basis. Let $\{\phi_\ell\}_{\ell=1}^k$ be the eigenvectors associated with the k smallest eigenvalues. Concatenating their real and imaginary parts yields a $2k$ -dimensional node positional encoding $\mathbf{z}_v = [\text{Re } \phi_1(v), \text{Im } \phi_1(v), \dots, \text{Re } \phi_k(v), \text{Im } \phi_k(v)]$ that smoothly rotates along edge directions and preserves orientation information. An optional lightweight two-layer MLP (MAGLAPNET) may further refine $\mathbf{z}_v$ before it is passed to the transformer encoder.

Multi- q Magnetic-Laplacian spectral encodings (Huang et al., 2024b). Building on the Magnetic-Laplacian $L_U^{(q)}$, fix a collection of Q distinct potentials $\{q_j\}_{j=1}^Q \subset [0, 1)$. For each q_j , compute the eigenvectors $\{\phi_\ell^{(q_j)}\}_{\ell=1}^k$ of

$$L_U^{(q_j)} = D_s - A_s \odot \exp(i\Theta^{(q_j)}), \quad \Theta_{uv}^{(q_j)} = 2\pi q_j (A_{uv} - A_{vu}),$$

associated with its k smallest eigenvalues. Define the $2k$ -dimensional embedding for node v at potential q_j by

$$\mathbf{z}_v^{(q_j)} = [\text{Re } \phi_1^{(q_j)}(v), \text{Im } \phi_1^{(q_j)}(v), \dots, \text{Re } \phi_k^{(q_j)}(v), \text{Im } \phi_k^{(q_j)}(v)].$$

Concatenating these across all Q potentials yields

$$\mathbf{z}_v = [\mathbf{z}_v^{(q_1)}, \dots, \mathbf{z}_v^{(q_Q)}] \in \mathbb{R}^{2kQ}.$$

For $Q \geq \lceil L/2 \rceil + 1$, this construction aggregates the phase responses at multiple “frequencies,” permitting exact recovery of all bidirectional walk counts up to length L and thereby strictly enhancing expressivity over any single- q scheme.

Directional random-walk encodings (Geisler et al., 2023). Let $T = AD_{\text{out}}^{-1}$ be the forward transition matrix and $R = A^\top D_{\text{in}}^{-1}$ the reverse transition matrix, where D_{out} and D_{in} are diagonal out- and in-degree matrices. Self-loops are added to sinks and sources to guarantee ergodicity. For each ordered node pair (u, v) and a fixed horizon k , collect the landing probabilities

$$[\mathbf{R}_{vu}^k, \dots, \mathbf{R}_{vu}, \mathbf{T}_{vu}, \dots, \mathbf{T}_{vu}^k] \quad \text{and} \quad \text{PPR}_{vu}, \quad (26)$$

where PPR denotes personalized PageRank scores. A two-layer MLP $f_{\text{rw}}^{(2)}$ embeds this pairwise feature vector, and a second MLP $f_{\text{rw}}^{(1)}$ aggregates over all senders u to yield a node-level positional encoding $\zeta(v|G)$. These encodings capture local directional structure through finite-step walks and global context via PageRank.

B. Deferred Proofs

Proposition 11 (Dune Expressivity (formal)). *Let $G = (V, E)$ be a finite directed graph with adjacency matrix $A \in \mathbb{R}^{n \times n}$. Let $\Pi(A)$ be the (fixed) skew-Hermitian Dune generator built from A . For a node v and a feature matrix X , define the self, out, and in multisets of embeddings as $S_v^{\text{self}}(X) = \{\{\mathbf{x}_v\}\}$, $S_v^{\text{out}}(X) = \{\{\mathbf{x}_u : u \in N_{\text{out}}(v)\}\}$, and $S_v^{\text{in}}(X) = \{\{\mathbf{x}_u : u \in N_{\text{in}}(v)\}\}$, where braces denote multisets (order ignored, multiplicities preserved). Fix a finite set of distinct step sizes $T = \{t_1, \dots, t_m\} \subset \mathbb{R}_{>0}$ and define the stacked multi- t preactivation*

$$\mathbf{Z}_T(X) := [\mathbf{X} \parallel (\mathbf{I} + t_1 \Pi(A))\mathbf{X} \parallel \dots \parallel (\mathbf{I} + t_m \Pi(A))\mathbf{X}],$$

so that $[\mathbf{Z}_T(X)]_v$ is the concatenation of $\mathbf{x}_v$ and the rows $[(\mathbf{I} + t_j \Pi(A))\mathbf{X}]_v$, $j = 1, \dots, m$. Assume the following multi-probe injectivity at each layer: for any feature matrix X , the map

$$v \mapsto [\mathbf{Z}_T(X)]_v$$

is an injective function of the pair of multisets $\{\{\mathbf{X}_u : u \in N_{\text{out}}(v)\}\}$ and $\{\{\mathbf{X}_u : u \in N_{\text{in}}(v)\}\}$ together with $\mathbf{X}_v$. Each linear map $\mathbf{W}^{(k)} : \mathbb{C}^{(m+1)d_k} \rightarrow \mathbb{C}^{d_{k+1}}$ and the nonlinearity σ are injective. From the Taylor expansion in equation 6, we use the first-order multi- t Dune update

$$\mathbf{X}^{(k+1)} = \sigma(\mathbf{Z}_T(\mathbf{X}^{(k)}) \mathbf{W}^{(k)}), \quad k = 0, \dots, K-1.$$

The directed 1-WL (D-1-WL) update colors nodes via an injective hash of $(c^{(k)}(v), \{\{c^{(k)}(u) : u \in N_{\text{out}}(v)\}\}, \{\{c^{(k)}(u) : u \in N_{\text{in}}(v)\}\})$. Initialize $\mathbf{X}^{(0)}$ so that $\mathbf{x}_v^{(0)} = \mathbf{x}_w^{(0)} \iff c^{(0)}(v) = c^{(0)}(w)$. Then for all $k \geq 0$ and all nodes v, w ,

$$\mathbf{x}_v^{(k)} = \mathbf{x}_w^{(k)} \iff c^{(k)}(v) = c^{(k)}(w). \quad (\star_k)$$

Proof of Proposition 11. We proceed by induction on k . Note that the base case ($k = 0$) holds by construction of $\mathbf{X}^{(0)}$. For the induction step ($k \rightarrow k+1$), assume $(\star_k)$ holds. We now prove both directions of $(\star_{k+1})$.

Forward direction. Assume $\mathbf{x}_v^{(k+1)} = \mathbf{x}_w^{(k+1)}$. Injectivity of σ and $\mathbf{W}^{(k)}$ yields

$$[\mathbf{Z}_T(\mathbf{X}^{(k)})]_v = [\mathbf{Z}_T(\mathbf{X}^{(k)})]_w.$$

By the multi-probe injectivity assumption, this equality forces the (self, out-, in-) multisets built from $\mathbf{x}^{(k)}$ at v and w to be identical. Applying the induction hypothesis $(\star_k)$ nodewise converts these multiset equalities of features into multiset equalities of colors, i.e.

$$\begin{aligned} \{\{c^{(k)}(u) : u \in N_{\text{out}}(v)\}\} &= \{\{c^{(k)}(u) : u \in N_{\text{out}}(w)\}\}, \\ \{\{c^{(k)}(u) : u \in N_{\text{in}}(v)\}\} &= \{\{c^{(k)}(u) : u \in N_{\text{in}}(w)\}\}, \end{aligned}$$

and $c^{(k)}(v) = c^{(k)}(w)$. By the D-1-WL update rule (which hashes these three ingredients injectively), we conclude $c^{(k+1)}(v) = c^{(k+1)}(w)$. This proves the forward implication of $(\star_{k+1})$.

Reverse direction. Assume $c^{(k+1)}(v) = c^{(k+1)}(w)$. By the D-1-WL update rule, there is a color-preserving bijection between the multisets

$$\{\{c^{(k)}(u) : u \in N_{\text{out}}(v)\}\} \quad \text{and} \quad \{\{c^{(k)}(u) : u \in N_{\text{out}}(w)\}\},$$

and likewise for in-neighbors, and also $c^{(k)}(v) = c^{(k)}(w)$. By the induction hypothesis $(\star_k)$, equality of colors implies equality of features, so the paired bijections lift to

$$\{\{\mathbf{x}_u^{(k)} : u \in N_{\text{out}}(v)\}\} = \{\{\mathbf{x}_u^{(k)} : u \in N_{\text{out}}(w)\}\},$$

and similarly for in-neighbors, with $\mathbf{x}_v^{(k)} = \mathbf{x}_w^{(k)}$. By the multi-probe injectivity assumption, these equalities of (in/out/self) multisets imply

$$[\mathbf{Z}_T(\mathbf{X}^{(k)})]_v = [\mathbf{Z}_T(\mathbf{X}^{(k)})]_w.$$

Applying the injective linear map $\mathbf{W}^{(k)}$ and the injective nonlinearity σ preserves equality, hence $\mathbf{x}_v^{(k+1)} = \mathbf{x}_w^{(k+1)}$. This proves the reverse implication of $(\star_{k+1})$.

Conclusion. By induction, $(\star_K)$ holds. Hence after K layers, Dune produces equal node embeddings if and only if D-1-WL assigns equal colors. With an injective graph-level readout, Dune distinguishes exactly the same pairs of nodes (and graphs) as K rounds of D-1-WL. $\square$

We note that the above argument closely resembles (Rossi et al., 2024). For more information on the directed Weisfeiler-Leman test and on possible choices for injective activation functions σ , refer to appendix D in their paper.

Proof of Proposition 6. Fix $t > 0$ and set $U := \exp(t \Pi(A))$. A Dune layer updates the features by

$$\mathbf{X}^{(k+1)} = U \mathbf{X}^{(k)} \mathbf{W}^{(k)},$$

where each trainable weight matrix $\mathbf{W}^{(k)} \in U(d)$ is unitary. Because left- and right-multiplication by unitary matrices preserve the Frobenius norm, $\|\mathbf{X}^{(k+1)}\|_F = \|\mathbf{X}^{(k)}\|_F$, so $\|\mathbf{X}^{(k)}\|_F = \|\mathbf{X}^{(0)}\|_F$ for all $k \geq 0$.

For any nonzero feature matrix $\mathbf{X} \in \mathbb{C}^{n \times d}$, define the normalized Dirichlet energy as before

$$E(\bar{\mathbf{X}}) = E\left(\frac{\mathbf{X}}{\|\mathbf{X}\|_F}\right) = \frac{\text{Tr}(\mathbf{X}^\dagger H_{\text{dir}} \mathbf{X})}{\|\mathbf{X}\|_F^2}. \quad (*)$$

Since H_{dir} is Hermitian, its Rayleigh quotient is bounded by its extremal eigenvalues:

$$\lambda_{\min}(H_{\text{dir}}) \leq \frac{\text{Tr}(\mathbf{X}^\dagger H_{\text{dir}} \mathbf{X})}{\|\mathbf{X}\|_F^2} \leq \lambda_{\max}(H_{\text{dir}}), \quad \forall \mathbf{X} \neq 0. \quad (**)$$

Let $\bar{E}_k := E(\bar{\mathbf{X}}^{(k)})$ be the normalized Dirichlet energy at layer k . Using $\mathbf{X}^{(k+1)} = U \mathbf{X}^{(k)} \mathbf{W}^{(k)}$ and unitarity of $\mathbf{W}^{(k)}$,

$$\bar{E}_{k+1} = \frac{\text{Tr}((\mathbf{X}^{(k)})^\dagger U^\dagger H_{\text{dir}} U \mathbf{X}^{(k)})}{\|\mathbf{X}^{(k)}\|_F^2},$$

because $(W^{(k)})^\dagger W^{(k)} = I$. Moreover, $U^\dagger H_{\text{dir}} U$ is unitarily similar to H_{dir} , so it has the same eigenvalues; hence the bounds in (**) hold for every $\bar{E}_k$.

Define

$$D_k := \bar{E}_k - \lambda_{\min}(H_{\text{dir}}) \geq 0.$$

Let $K := \{U^k : k \geq 0\} \subset U(n)$, which is compact. Consider the continuous map

$$f : K \rightarrow \mathbb{R}, \quad f(V) = \frac{\text{Tr}((X^{(0)})^\dagger V^\dagger H_{\text{dir}} V X^{(0)})}{\|X^{(0)}\|_F^2}.$$

Let $M^* := \max_{V \in K} f(V)$ and $m^* := \min_{V \in K} f(V)$ (both attained). If $X^{(0)}$ does not minimize the normalized Dirichlet energy, i.e. is not contained in the $\lambda_{\min}(H_{\text{dir}})$ -eigenspace, then we cannot have $M^* = \lambda_{\min}(H_{\text{dir}})$. Indeed, if $M^* = \lambda_{\min}(H_{\text{dir}})$ then $f(V) \equiv \lambda_{\min}(H_{\text{dir}})$ on K , and since $I \in K$ this would give $f(I) = \lambda_{\min}(H_{\text{dir}})$, forcing $X^{(0)}$ to lie in the minimizing eigenspace—a contradiction. Hence $M^* > \lambda_{\min}(H_{\text{dir}})$.

Pick $V^* \in K$ with $f(V^*) \geq \lambda_{\min}(H_{\text{dir}}) + \delta$ for some $\delta > 0$, and extract a subsequence $U^{k_\ell} \rightarrow V^*$. By continuity,

$$D_{k_\ell} = \bar{E}_{k_\ell} - \lambda_{\min}(H_{\text{dir}}) \rightarrow f(V^*) - \lambda_{\min}(H_{\text{dir}}) \geq \delta. \quad (***)$$

Now suppose, toward contradiction, that the model oversmooths, i.e. there exist $c > 0$ and $k_0 > 0$ such that for all $k \geq k_0$,

$$\bar{E}_k - \lambda_{\min}(H_{\text{dir}}) \leq e^{-ck} (\bar{E}_0 - \lambda_{\min}(H_{\text{dir}})).$$

Then in particular $D_k \leq e^{-ck} D_0$ for all $k \geq k_0$, so along the subsequence $k_\ell \rightarrow \infty$ we would have $D_{k_\ell} \rightarrow 0$, contradicting (***), which gives $D_{k_\ell} \geq \delta$ for all sufficiently large ℓ . Therefore no such c, k_0 exist, and the Dune network does not oversmooth. $\square$

Proof of Proposition 7. Note that since S is symmetric, DiGCN is equivalent to an undirected linear GCN on the symmetrized graph. Exponential oversmoothing therefore follows immediately from standard undirected results, for example (Cai & Wang, 2020). The rate of decay is controlled by the spectral gap (here upper-bounded by $1 - \alpha$). The additional requirement that $\inf_{k \geq 0} \|\Pi_{\min} X^{(0)} B_k\|_F > 0$ ensures that the stationary component is not sent to zero by the feature maps. $\square$

We would also like to refer the reader to (Chen et al., 2025), who show oversmoothing for these types of graphs under looser restrictions. Our proposition is only meant to illustrate an example for a specific architecture.

Proposition 12 (Dune captures information about the walk profiles (formal).). *Let G be a directed graph with adjacency A . Fix complex scalars c_+, c_- and set $\Pi(A) = c_+ A + c_- A^\top$. Consider a depth- K Dune stack in which each layer uses the order- T Taylor truncation of the matrix exponential with a fixed step $t > 0$. Then the pre-activations at depth K lie in $\text{span}\{\Pi(A)^\ell X^{(0)} B_\ell : 0 \leq \ell \leq KT\}$ for some matrices B_ℓ depending on the weights and nonlinearity. Consequently, the contribution from a source node u to a target node v at depth K is a weighted sum of bidirectional walk counts $\{\Phi_{u,v}(\ell, k)\}_{\ell \leq KT}$, where $\Phi_{u,v}(\ell, k)$ counts length- ℓ walks with exactly k forward and $\ell - k$ backward steps:*

$$(\Pi(A)^\ell)_{v,u} = \sum_{k=0}^{\ell} \gamma_{\ell,k} \Phi_{u,v}(\ell, k), \quad 0 \leq \ell \leq KT, \quad (27)$$

for coefficients $\gamma_{\ell,k}$ determined by $c_\pm$, the Taylor coefficients $t^\ell/\ell!$, and the network parameters.

Proof of Proposition 12 One layer applies $P_T(\Pi)$ to the current features, so its pre-activation is a linear combination of $\{\Pi(A)^\ell X^{(r)} : 0 \leq \ell \leq T\}$. Composing K such layers multiplies on the left by degree- T polynomials K times; collecting terms shows every depth- K pre-activation is a linear combination of $\{\Pi(A)^\ell X^{(0)} : \ell \leq KT\}$. Right-side weights and the pointwise nonlinearity only alter the coefficient matrices B_ℓ ; they do not increase the left-multiplicative degree in $\Pi(A)$.

Finally, $\Pi(A) = c_+ A + c_- A^\top$ implies $\Pi(A)^\ell$ expands into words of length ℓ in $\{A, A^\top\}$. The (v, u) entry of any such word corresponds to a directed walk from u to v whose steps are “forward” when the factor is A and “backward” when it is $A^\top$; grouping words by the number k of forward steps yields the decomposition above with weights $\gamma_{\ell,k}$. Hence, depth K with order- T propagation aggregates weighted bidirectional walk counts with horizon $\ell \leq KT$. $\square$

A few comments are in order: first, on some directed graphs, this is enough to capture the exact walk profile. For example, on a directed acyclic graph (DAG), Dune’s statistic collapses to a single bidirectional count: fix nodes $u \rightarrow v$ with shortest directed distance $d(u, v)$. Any length- $d(u, v)$ walk from u to v must be **all forward** ($k = \ell = d(u, v)$); inserting any backward step would force at least two extra steps. Hence for $\ell = d(u, v)$ (and also for $\ell < d(u, v)$, where both sides are zero) the Dune aggregation reduces to a single term $\gamma_{\ell, \ell} \Phi_{u, v}(\ell, \ell)$. Hence this recovers the walk profile up to a scalar. This dependency on the topology of the input graph when it comes to capturing information about the bidirectional walk profile also shows empirically (compare DAGs and regular graphs in table 14).

Also, note that a non-unitary directed GNN such as Dir-GCN would require KT layers to recover the same information. Assuming that we want KT to be roughly the diameter of the graph, this quickly becomes problematic on larger graphs due to oversmoothing and exploding or vanishing gradients with non-unitary GNNs. Empirically, we find that this is indeed the case: Dir-GCN performs significantly worse than Dune at predicting shortest and longest directed path distances and fails completely at predicting the walk profile (see table 14).

Finally, we should also point out that the full bidirectional walk profiles can be recovered from the weighted sums of bidirectional walk counts that Dune aggregates using techniques introduced in (Kanatsoulis et al., 2025). We did not implement this in this paper though as it would go significantly beyond hybrid architectures on directed graphs.

C. Complexity

Complexity. Let $n = |V|$ denote the number of nodes. Forming the Hermitian projection $\Pi(A)$ requires computing $A^\dagger$ and combining it with A , which takes $\Theta(n^2)$ time. The exponential map is then approximated by a Taylor expansion of order T ,

$$\exp(\Pi(A)) \approx \sum_{j=0}^T \frac{1}{j!} \Pi(A)^j, \quad (28)$$

and evaluating this expansion explicitly as a matrix requires T successive dense matrix–matrix multiplications of cost $\Theta(n^3)$ each, giving an overall time complexity of $\Theta(Tn^3)$. In practice, when applying the exponential to node features $X \in \mathbb{C}^{n \times d}$ instead of computing the full matrix exponential, the cost becomes T multiplications of an $n \times n$ matrix with an $n \times d$ feature matrix, leading to $\Theta(Tn^2d)$ in the dense case or $\Theta(Tmd)$ when the adjacency is sparse with m edges.

D. Order of the Taylor Expansion, Runtime, and Memory Consumption

In the following, we evaluate Dune with various values of T , where T is the order of the Taylor expansion used to compute the exponential map.

	Dune (T=2)	Dune (T=4)	Dune (T=6)	Dune (T=8)	Dune (T=10)
Citeseer	94.12 $\pm$ 0.63	94.39 $\pm$ 0.83	93.71 $\pm$ 0.64	92.77 $\pm$ 0.73	92.94 $\pm$ 0.78
OGBN-Arxiv	70.79 $\pm$ 0.39	72.16 $\pm$ 0.28	71.63 $\pm$ 0.32	71.24 $\pm$ 0.35	71.11 $\pm$ 0.44
Questions	77.94 $\pm$ 0.80	78.46 $\pm$ 0.83	79.27 $\pm$ 0.76	80.33 $\pm$ 0.79	80.45 $\pm$ 0.88

Table 4. Dune accuracy (ROC-AUC) with different values of T . All other hyperparameters kept fixed and the values reported in the original manuscript.

	Dir-GCN	Dir-GAT	Dune (T=2)	Dune (T=4)	Dune (T=6)	Dune (T=8)	Dune (T=10)
Citeseer	146.29	276.69	216.45	249.50	282.55	315.73	348.78
OGBN-Arxiv	4063.40	17978.97	14062.54	15390.09	16718.63	18046.28	19374.21
Questions	1241.49	3254.66	2760.74	3144.82	3528.46	3912.92	4296.32

Table 5. Peak memory consumption (MB) with 4 layers and hidden dim. 256. Lower is better.

Based on tables 5 and 6, we note that for most values of T , the runtime and peak memory consumption of Dune is comparable with the well-established Dir-GAT model, and that excessively large values of T confer small, if any, performance boosts (see table 4).

	Dir-GCN	Dir-GAT	Dune (T=2)	Dune (T=4)	Dune (T=6)	Dune (T=8)	Dune (T=10)
Citeseer	0.00451	0.00851	0.00593	0.00987	0.01350	0.01880	0.02791
OGBN-Arxiv	0.05994	0.10627	0.14925	0.22573	0.33003	0.54054	0.86207
Questions	0.01688	0.02514	0.01954	0.02728	0.04143	0.05327	0.09671

Table 6. Runtime per training epoch (seconds) with 4 layers and hidden dim. 256. Lower is better.

E. Alternative Definitions for Unitary Convolutions on Directed Graphs

In this subsection, we present two alternative ways to define a unitary graph neural network on directed graphs. We did not implement these variants, but believe that they might serve as inspiration for future work.

E.1. Lie Dune

Definition (Directed Lie-UniConv). Let $G = (V, E)$ be a directed graph on n nodes with (possibly asymmetric) adjacency matrix $A \in \mathbb{R}^{n \times n}$. Define the skew-Hermitian projection

$$\Pi(A) = \frac{i}{4}(A + A^\top) + \frac{1}{4}(A - A^\top) \in \mathfrak{u}(n),$$

as before and let $W \in \mathbb{C}^{d \times d}$ satisfy $W + W^\dagger = 0$. Define the operator

$$g_{\text{conv}}(X) = \Pi(A) X W.$$

Then the *directed Lie UniConv* layer is

$$f_{\text{DLie}}(X) = \exp(g_{\text{conv}})(X) = \sum_{k=0}^{\infty} \frac{1}{k!} g_{\text{conv}}^{(k)}(X),$$

where

$$g_{\text{conv}}^{(0)}(X) = X, \quad g_{\text{conv}}^{(k)}(X) = \Pi(A)(g_{\text{conv}}^{(k-1)}(X)) W \quad (k \geq 1).$$

We did not implement this, as it exponentiates W by default. If W is skew-hermitian, this results in a unitary weight matrix, which (Kiani et al., 2024) did not always find beneficial, especially on node-level tasks.

E.2. Dir-GCN Analogue

Definition (Directed Separable Unitary Graph Convolution). Let $G = (V, E)$ be a directed graph on n nodes with adjacency matrix $A \in \mathbb{R}^{n \times n}$, which we split into “out-” and “in-” adjacency:

$$A_{\rightarrow} = A, \quad A_{\leftarrow} = A^\top.$$

As in Section 4.1, project each into $\mathfrak{u}(n)$ via

$$\Pi(A) = \frac{i}{4}(A + A^\top) + \frac{1}{4}(A - A^\top),$$

so that $\Pi(A_{\rightarrow}) = \Pi(A)$ and $\Pi(A_{\leftarrow}) = \Pi(A)^\top$ are both skew-Hermitian. Given a diffusion time $t > 0$, a convex-combination weight $\alpha \in [0, 1]$, and learnable (unitary) feature-mixers $U_{\rightarrow}, U_{\leftarrow} \in U(d)$, the *directed separable UniConv* layer is

$$f_{\text{DirUConv}}(X; A) = \alpha \exp(\Pi(A)t) X U_{\rightarrow} + (1 - \alpha) \exp(\Pi(A)^\top t) X U_{\leftarrow} \in \mathbb{C}^{n \times d},$$

which in the small- t limit recovers the usual Dir-GCN-style split aggregation $\alpha A_{\rightarrow} X + (1 - \alpha) A_{\leftarrow} X$

We did not implement this as it is by construction computationally more costly to compute than our proposed Dune while performing essentially the same convolution operation for small t and appropriate α .

Model	Cora	Citeseer	OGBN-Arxiv	Arxiv-Year	Roman E.	Questions
GCN	84.37 $\pm$ 1.52	93.37 $\pm$ 0.22	68.39 $\pm$ 0.11	46.28 $\pm$ 0.39	56.23 $\pm$ 0.37	76.09 $\pm$ 1.27
SAGE	86.01 $\pm$ 1.56	94.15 $\pm$ 0.61	67.78 $\pm$ 0.17	44.05 $\pm$ 0.22	72.05 $\pm$ 0.41	76.44 $\pm$ 0.62
GCN-In	84.53 $\pm$ 1.65	93.86 $\pm$ 0.24	63.84 $\pm$ 0.25	52.56 $\pm$ 0.51	52.26 $\pm$ 0.42	76.30 $\pm$ 1.53
Sage-In	85.26 $\pm$ 1.49	93.92 $\pm$ 0.67	65.77 $\pm$ 0.22	53.38 $\pm$ 0.29	71.28 $\pm$ 0.46	77.13 $\pm$ 0.71
GCN-Out	85.07 $\pm$ 1.76	93.33 $\pm$ 0.35	24.04 $\pm$ 0.67	51.90 $\pm$ 0.44	44.14 $\pm$ 0.67	74.96 $\pm$ 1.41
Sage-Out	85.47 $\pm$ 1.44	92.80 $\pm$ 0.58	52.01 $\pm$ 0.39	49.76 $\pm$ 0.38	79.76 $\pm$ 0.43	75.67 $\pm$ 0.85
Dir-GCN	84.45 $\pm$ 1.69	93.44 $\pm$ 0.59	66.66 $\pm$ 0.12	64.08 $\pm$ 0.26	84.54 $\pm$ 0.71	75.94 $\pm$ 1.36
Dir-Sage	85.84 $\pm$ 2.09	94.14 $\pm$ 0.65	65.14 $\pm$ 0.13	61.76 $\pm$ 0.10	91.23 $\pm$ 0.32	77.43 $\pm$ 0.85
Dune	87.82 $\pm$ 1.14	94.39 $\pm$ 0.83	72.16 $\pm$ 0.28	65.86 $\pm$ 0.10	92.58 $\pm$ 0.22	80.45 $\pm$ 0.88

Table 7. Additional MPNN baselines on directed node classification datasets. GCN-In and Sage-In only aggregate information from incoming edges; GCN-Out and Sage-Out only aggregate information from outgoing edges.

Model	Cora	Citeseer	Roman Empire	Questions
GraphGPS (GIN)	82.48 $\pm$ 0.93	92.27 $\pm$ 1.07	82.52 $\pm$ 0.66	71.93 $\pm$ 1.54
Exphormer (GIN)	82.69 $\pm$ 1.27	91.42 $\pm$ 1.30	83.28 $\pm$ 0.58	72.74 $\pm$ 1.23
GraphGPS (bidirected-GIN)	83.45 $\pm$ 1.19	92.12 $\pm$ 1.22	87.10 $\pm$ 0.74	74.17 $\pm$ 1.46
Exphormer (bidirected-GIN)	83.68 $\pm$ 1.35	92.39 $\pm$ 1.37	87.75 $\pm$ 0.80	73.84 $\pm$ 1.32
DUNE	87.82 $\pm$ 1.14	94.39 $\pm$ 0.83	92.58 $\pm$ 0.22	80.45 $\pm$ 0.88

Table 8. Dune comparison with transformer-based models on directed node classification datasets.

F. Additional Experiments

F.1. Additional node classification baselines

unitary neural networks were introduced specifically to guarantee stable gradients and no oversmoothing without skip connections or layer normalization (see *Related Work* section). Here, we report additional results on the role of these architecture choices. The number of parameters is fixed between models. As expected, GCN and Dir-GCN benefit from both skip connections and normalization, while the effect for DUNE is generally not statistically significant. This aligns with our prior. We used skip connections and layer normalization in all other experiments in this paper unless explicitly stated otherwise.

Model	Skip + Norm.	Skip only	Norm. only	Neither
GCN	56.23 $\pm$ 0.37	55.96 $\pm$ 0.46	55.02 $\pm$ 0.59	54.68 $\pm$ 0.66
Dir-GCN	84.54 $\pm$ 0.71	83.61 $\pm$ 0.68	83.17 $\pm$ 0.82	83.03 $\pm$ 0.95
DUNE	92.58 $\pm$ 0.22	92.45 $\pm$ 0.26	92.36 $\pm$ 0.39	92.12 $\pm$ 0.44

Table 9. Ablation on the role of skip connections and normalization on the Roman Empire dataset (we report accuracy, higher is better).

Model	Skip + Norm.	Skip only	Norm. only	Neither
GCN	76.09 $\pm$ 1.27	74.58 $\pm$ 1.34	73.87 $\pm$ 1.38	73.19 $\pm$ 1.46
Dir-GCN	75.94 $\pm$ 1.36	75.03 $\pm$ 1.39	74.82 $\pm$ 1.42	74.21 $\pm$ 1.48
DUNE	80.45 $\pm$ 0.88	80.10 $\pm$ 0.93	79.92 $\pm$ 0.98	79.65 $\pm$ 1.04

Table 10. Ablation on the role of skip connections and normalization on the Questions dataset (we report ROC-AUC, higher is better).

F.2. Directed graph regression

Model	DSP	LUT
UniGCN	2.609 ± 0.113	2.030 ± 0.102
DUNE, no edges	2.316 ± 0.104	1.884 ± 0.095
DUNE, linear edges	2.204 ± 0.091	1.753 ± 0.081
DUNE, MLP edges	2.219 ± 0.088	1.738 ± 0.086
SAT (UniGCN)	2.891 ± 0.149	2.315 ± 0.126
SAT (DUNE, no edges)	2.728 ± 0.137	2.179 ± 0.113
SAT (DUNE, linear edges)	2.612 ± 0.111	2.021 ± 0.093
SAT (DUNE, MLP edges)	2.590 ± 0.120	2.046 ± 0.098

Table 11. Graph regression results on the HLS datasets. Lower is better. “Linear” and “MLP” refer to the way the edge feature map was parametrized.

Model	PE method	Gain	BW	PM	DSP	LUT
Undirected-GIN	None	0.462 ± 0.023	4.791 ± 0.122	1.380 ± 0.030	3.405 ± 0.204	2.818 ± 0.190
	Lap	0.416 ± 0.021	4.321 ± 0.084	1.127 ± 0.020	2.662 ± 0.187	1.925 ± 0.059
	Maglap-1q (best q)	0.398 ± 0.025	4.281 ± 0.085	1.113 ± 0.022	2.614 ± 0.098	2.010 ± 0.082
	Maglap-Multi-q	0.389 ± 0.017	4.175 ± 0.115	1.137 ± 0.004	2.582 ± 0.133	1.976 ± 0.089
Bidirected-GIN	None	0.466 ± 0.027	4.879 ± 0.118	1.318 ± 0.030	2.637 ± 0.170	1.814 ± 0.076
	Lap	0.391 ± 0.007	4.153 ± 0.160	1.135 ± 0.035	2.267 ± 0.126	1.786 ± 0.072
	Maglap-1q (best q)	0.383 ± 0.002	4.113 ± 0.052	1.099 ± 0.020	2.256 ± 0.144	1.768 ± 0.090
	Maglap-Multi-q	0.371 ± 0.008	4.051 ± 0.139	1.116 ± 0.012	2.207 ± 0.185	1.735 ± 0.096
UniGCN	None	0.391 ± 0.018	4.475 ± 0.096	1.163 ± 0.027	2.609 ± 0.113	2.030 ± 0.102
	Lap	0.377 ± 0.016	4.092 ± 0.088	1.076 ± 0.028	2.481 ± 0.109	1.902 ± 0.063
	Maglap-1q (best q)	0.376 ± 0.012	4.086 ± 0.093	1.092 ± 0.032	2.496 ± 0.102	1.933 ± 0.068
	Maglap-Multi-q	0.389 ± 0.022	4.162 ± 0.125	1.106 ± 0.024	2.474 ± 0.094	1.895 ± 0.074
Dune	None	0.380 ± 0.011	4.154 ± 0.083	1.128 ± 0.024	2.204 ± 0.091	1.753 ± 0.081
	Lap	0.376 ± 0.007	4.102 ± 0.080	1.079 ± 0.027	2.190 ± 0.102	1.742 ± 0.075
	Maglap-1q (best q)	0.378 ± 0.010	4.057 ± 0.064	1.106 ± 0.030	2.195 ± 0.111	1.778 ± 0.096
	Maglap-Multi-q	0.383 ± 0.008	4.013 ± 0.078	1.081 ± 0.019	2.113 ± 0.094	1.682 ± 0.063

Table 12. Graph regression results using message-passing architectures. Reported are mean RMSE (lower is better) and standard deviation over 10 independent runs.

Model	PE method	Gain	BW	PM	DSP	LUT
GPS (UniGCN)	None	0.398 ± 0.014	4.428 ± 0.122	1.201 ± 0.033	2.781 ± 0.106	2.153 ± 0.084
	Lap	0.403 ± 0.028	4.334 ± 0.108	1.205 ± 0.030	2.714 ± 0.095	2.118 ± 0.064
	Maglap-1q (best q)	0.410 ± 0.017	4.365 ± 0.113	1.228 ± 0.036	2.722 ± 0.113	2.081 ± 0.105
	Maglap-Multi-q	0.407 ± 0.020	4.352 ± 0.102	1.194 ± 0.038	2.689 ± 0.120	2.052 ± 0.077
GPS (Dune)	None	0.403 ± 0.012	4.253 ± 0.128	1.126 ± 0.028	2.707 ± 0.088	2.076 ± 0.072
	Lap	0.405 ± 0.013	4.405 ± 0.113	1.130 ± 0.027	2.635 ± 0.131	2.029 ± 0.070
	Maglap-1q (best q)	0.456 ± 0.021	4.492 ± 0.110	1.142 ± 0.022	2.660 ± 0.118	2.003 ± 0.086
	Maglap-Multi-q	0.434 ± 0.018	4.387 ± 0.116	1.120 ± 0.020	2.608 ± 0.102	1.975 ± 0.062

Table 13. Graph regression results using GraphGPS with a unitary backbone. Reported are mean RMSE (lower is better) and standard deviation over 10 independent runs.

F.3. Training dynamics and improved generalization

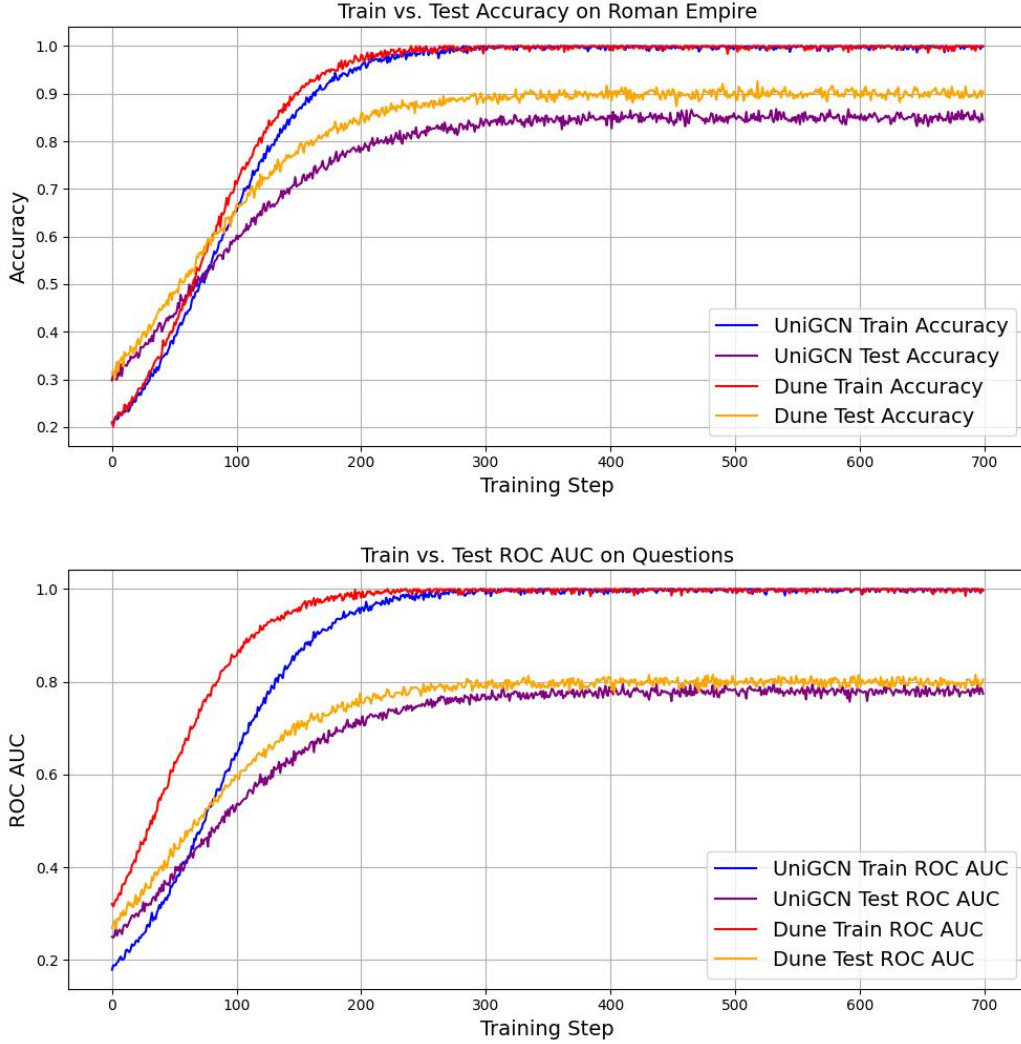


Figure 5. Generalization gaps of UniGCN and Dune on Roman Empire and Questions.

F.4. Positional Encodings Playground

PE method	Directed Acyclic Graph			Regular Directed Graph		
	<i>spd</i>	<i>lpd</i>	<i>wp(4, ·)</i>	<i>spd</i>	<i>lpd</i>	<i>wp(4, ·)</i>
Lap	0.355 ± 0.001	0.655 ± 0.002	0.326 ± 0.001	2.066 ± 0.005	1.920 ± 0.000	0.452 ± 0.001
SVD	0.727 ± 0.001	0.912 ± 0.001	0.721 ± 0.000	2.261 ± 0.002	2.122 ± 0.007	0.755 ± 0.000
MagLap-1q (best q)	0.124 ± 0.002	0.432 ± 0.004	0.040 ± 0.001	1.533 ± 0.007	1.493 ± 0.003	0.132 ± 0.000
MagLap-Multi-q	0.016 ± 0.000	0.185 ± 0.036	0.002 ± 0.000	0.546 ± 0.068	1.100 ± 0.007	0.074 ± 0.001
Dir-GCN	0.391 ± 0.014	0.411 ± 0.012	3.529 ± 0.168	1.960 ± 0.074	3.508 ± 0.361	5.913 ± 0.646
Dune	0.013 ± 0.006	0.134 ± 0.009	0.241 ± 0.082	0.804 ± 0.072	1.419 ± 0.085	0.535 ± 0.133

Table 14. RMSE results over 3 random seeds for node-pair distance prediction and walk profile reconstruction (lower is better).

Results on synthetic datasets can be found in Table 14. Here, we follow (Geisler et al., 2023) and (Huang et al., 2024b) and create directed acyclic or regular graphs. We then formulate a regression task by asking a shallow transformer to predict the shortest (*‘spd’*) or longest (*‘lpd’*) walk distance between two nodes, as well as the complete walk profile (*‘wp’*). The

geometric information required for this is provided by either one of four positional encodings or by the forward pass of specifically trained MP-GNN. As Table 14 shows, the model using Dune to derive its geometric information outperforms Lap, SVD, and MagLap-1q positional encodings and the Dir-GCN model while beating MagLap-Multi-q on some datasets.

F.5. Message-passing on undirected Graphs

Model	Zinc Test MAE ↓	Peptides-func Test AP ↑	Peptides-struct Test MAE ↓	Coco Test F1 ↑	Pascal Test F1 ↑
GCN	0.367 ± 0.011	0.6860 ± 0.0050	0.2460 ± 0.0007	0.1338 ± 0.0007	0.2078 ± 0.0031
GINE	0.326 ± 0.041	0.6621 ± 0.0067	0.2473 ± 0.0017	0.2125 ± 0.0009	0.2718 ± 0.0054
GatedGCN	0.282 ± 0.015	0.6765 ± 0.0047	0.2477 ± 0.0009	0.2922 ± 0.0018	0.3880 ± 0.0040
UniGCN	0.142 ± 0.022	0.7072 ± 0.0035	0.2425 ± 0.0009	0.2852 ± 0.0016	0.3516 ± 0.0070
Dune	0.093 ± 0.018	0.7114 ± 0.0038	0.2410 ± 0.0011	0.3019 ± 0.0022	0.3675 ± 0.0062
Dune(MLP)	0.091 ± 0.020	0.7103 ± 0.0042	0.2416 ± 0.0008	0.3032 ± 0.0020	0.3658 ± 0.0065

Table 15. Performance comparison between message-passing architecture on undirected graph datasets (Gómez-Bombarelli et al., 2018; Dwivedi et al., 2022) over 3 random seeds. Dune(MLP) uses a two-layer MLP with gelu activation to process edge features, as opposed to standard Dune’s linear layer. We fix the model depth at 16 layers for UniGCN, Dune, and Dune (MLP).

F.6. Experiments with deeper models

Model	4 Layers	8 Layers	16 Layers	32 Layers
Bidirected-GIN	2.637 ± 0.170	2.348 ± 0.143	2.466 ± 0.158	2.695 ± 0.135
UniGCN	2.609 ± 0.113	2.552 ± 0.109	2.583 ± 0.127	2.634 ± 0.117
Dune	2.204 ± 0.091	2.145 ± 0.096	2.112 ± 0.088	2.086 ± 0.084

Table 16. RMSE vs depth on the DSP dataset. Dune performance improves while UniGCN and Bidirected-GIN plateau or drop.

Model	4 Layers	8 Layers	16 Layers	32 Layers
Bidirected-GIN	1.814 ± 0.076	1.765 ± 0.084	1.869 ± 0.091	1.927 ± 0.080
UniGCN	2.030 ± 0.102	1.974 ± 0.095	2.050 ± 0.078	2.061 ± 0.089
Dune	1.753 ± 0.081	1.712 ± 0.072	1.684 ± 0.075	1.645 ± 0.068

Table 17. RMSE vs depth on the LUT dataset. Dune performance improves while UniGCN and Bidirected-GIN plateau or drop.

Model	8 Layers	16 Layers	32 Layers	64 Layers
GCN	0.6864 ± 0.0050	0.6907 ± 0.0047	0.6429 ± 0.0054	Diverges
UniGCN	0.7072 ± 0.0035	0.7096 ± 0.0044	0.7125 ± 0.0040	0.7163 ± 0.0047
Dune	0.7108 ± 0.0042	0.7114 ± 0.0038	0.7183 ± 0.0044	0.7221 ± 0.0050

Table 18. AP vs depth on the Peptides-func dataset.

Model	8 Layers	16 Layers	32 Layers	64 Layers
GCN	0.2062 ± 0.0034	0.2037 ± 0.0038	0.1374 ± 0.0047	Diverges
UniGCN	0.3482 ± 0.0058	0.3516 ± 0.0070	0.3628 ± 0.0067	0.3673 ± 0.0074
Dune	0.3641 ± 0.0047	0.3675 ± 0.0062	0.3789 ± 0.0060	0.3870 ± 0.0072

Table 19. F1 vs depth on the Pascal dataset.

E.7. Hybrid Architectures on undirected Graphs

Model	Zinc Test MAE ↓	Peptides-func Test AP ↑	Peptides-struct Test MAE ↓	Coco Test F1 ↑	Pascal Test F1 ↑
GPS(GINE)	0.083 ± 0.007	0.6534 ± 0.0091	0.2509 ± 0.0014	0.3884 ± 0.0055	0.4440 ± 0.0065
GPS(UniGCN)	0.101 ± 0.014	0.6453 ± 0.0086	0.2528 ± 0.0017	0.3629 ± 0.0067	0.4186 ± 0.0092
GPS(Dune)	0.069 ± 0.010	0.6718 ± 0.0097	0.2477 ± 0.0015	0.3952 ± 0.0061	0.4498 ± 0.0071

Table 20. Performance comparison between GraphGPS with different backbones on undirected graph datasets over 3 random seeds.

G. Experimental Details

G.1. Hyperparameter Search Procedure

We carried out a per-dataset grid search, varying the learning rate in $\{0.001, 0.0005, 0.0001\}$, hidden dimension in $\{64, 128, 256, 512\}$, number of layers in $\{6, 8, 12, 16, 20, 24\}$, and dropout rate in $\{0, 0.2, 0.4, 0.5, 0.7\}$. Each configuration was trained using the Adam optimizer for up to 10 000 epochs with early stopping on validation accuracy (patience = 500) and no additional regularization.

G.2. Hyperparameter Configurations

Model	Cora	Citeseer	OGBN-Arxiv	Arxiv-Year	SNAP-Patents	Roman E.	Questions
Learning Rate	0.001	0.001	0.0005	0.0005	0.001	0.001	0.001
Hidden Dim.	64	512	512	512	64	512	256
Num. Layers	8	8	6	6	12	24	16
Dropout	0.5	0.7	0	0	0	0.4	0.4

Table 21. Optimal hyperparameters on node classification datasets with Dune.

Model	Gain	BW	PM	DSP	LUT
Learning Rate	0.001	0.001	0.001	0.004	0.004
Hidden Dim.	54	78	54	78	62
Num. Layers	3	3	3	4	3
Dropout	0.2	0.3	0.2	0.1	0.1

Table 22. Optimal hyperparameters on graph regression datasets with Dune.

G.3. Datasets

Dataset	# Nodes	# Edges	# Feat.	# C	Unidirectional Edges	Edge hom.
Citeseer-Full	4,230	5,358	602	6	99.61%	0.949
Cora-ML	2,995	8,416	2,879	7	96.84%	0.792
OGBN-Arxiv	169,343	1,166,243	128	40	99.27%	0.655
Arxiv-Year	169,343	1,166,243	128	40	99.27%	0.221
SNAP-Patents	2,923,922	13,975,791	269	5	99.98%	0.218
Roman-Empire	22,662	44,363	300	18	65.24%	0.050

Table 23. Statistics of the node classification datasets used in this paper.

G.4. Complex-to-real readout

The Dune propagation operator produces complex-valued hidden states, whereas the prediction tasks considered in this work require real-valued outputs. In line with prior work on complex-valued graph models, we experimented with several projections from complex-valued features to real-valued features, including the modulus, the real component, and the imaginary component of the final representation. We found that using the real component worked best in our experiments, although differences between these choices were generally small. Unless otherwise stated, this is the readout convention used throughout. This projection is separate from the Dune propagation operator itself: the expressivity result in Proposition

3 concerns the complex-valued update with an injective readout, not every possible real-valued projection. In particular, taking only the real component can discard information that remains purely imaginary at the final layer, while exposing both real and imaginary components would remove this possible projection bottleneck without changing the propagation mechanism.

H. Hardware Configuration

All experiments in this paper were run on a single Nvidia A100 GPU with 80GB RAM and CUDA version 12.9.